2026년도 졸업논문

UWB를 이용한 스마트 카트

한국공학대학교

전자공학부

박성훈, 이지현, 정상현, 최어진

UWB를 이용한 스마트 카트

Smart Cart System Using UWB-Based Distance Measurement

2026年　　2月　20日

한국공학대학교

전자공학부

박성훈, 이지현, 정상현, 최어진

UWB를 이용한 스마트 카트

指導敎授 이 재 명 

이 論文을 졸업논문으로 提出함.

2026年 2月 20日

한국공학대학교

전자공학부

박성훈, 이지현, 정상현, 최어진

목차

그림 목차 ····· 6

국문 요약 ····· 6

영문 요약 ····· 6

제 1 장. 서론 ····· 9

1.1. 개발 배경 ····· 9

1.2. 개발 목표 ····· 9

제 2 장. 본론 ····· 11

2.1. 전체 시스템 구성도 ····· 11

2.2. 하드웨어 구성 ····· 12

(1) UWB 기반 위치 추정 모듈 ····· 13

(2) 중앙처리 제어부 ····· 13

(3) 모터 구동부 ····· 14

(4) 초음파 센서부 ····· 15

(5) 전원 공급부 ····· 16

2.3. 통신 시스템 ····· 17

(1) UWB 거리 측정 통신 구조 ····· 18

(2) Raspberry Pi 내부 통신 (UART) ····· 20

(3) Raspberry Pi와 스마트폰 간 통신 (TCP/IP) ····· 21

2.4. 소프트웨어 및 알고리즘 ····· 23

(1) ESP32 Tag/Anchor 펌웨어 구조 ····· 23

(2) UWB 거리 데이터 후처리 ····· 29

(3) 보정 알고리즘 ····· 31

(4) 두 앵커 기반 삼각측량 알고리즘 ····· 33

(5) 카트 추종 제어 알고리즘 ····· 35

(6) 장애물 감지 및 안전 제어 ····· 37

(7) 네트워크 기반 명령 및 데이터 흐름 ····· 38
(8) 상태 기반 제어 ····· 40
(9) 안드로이드 앱 설계 ····· 43
2.5. 개발 결과 및 성능 평가 ····· 46
(1) UWB 거리 데이터 필터링 성능 ····· 46
(2) 거리 오차 보정 성능 ····· 49
(3) 삼각측량 기반 위치 추정 정확도 ····· 50
(4) 통합 주행 제어 및 시스템 연동 검증 ····· 51

제 3 장. 결론 ····· 54
3.1. 결론 ····· 54
3.2. 향후 발전 ····· 55

참고문헌 ····· 56
부록 ····· 57

<제목 차례>

<그림 2.1.1> 시스템 구성도 ····12

<그림 2.2.1> 하드웨어 장착 카트 실물 ····13

<그림 2.2.2> 하드웨어 전체 구성도 ····13

<그림 2.2.3 (A) ESP32 (B) DWM1000 > ····14

<그림 2.2.4> Raspberry Pi 4 Model B ····15

<그림 2.2.5> JGA25-370 모터 ····16

<그림 2.2.6> L298N 모터 드라이버 ····16

<그림 2.2.7> 초음파 센서 HC-SR04 ····17

<그림 2.2.8> 12V 리튬이온 배터리 팩 ····17

<그림 2.3.1> 3계층 통신 구조 ····18

<그림 2.3.2> 전체 시스템 통신 구조 ····19

<그림 2.3.3> ADS-TWR 통신 ····19

<그림 2.3.4> 라즈베리파이 내부 연결 장치 ····22

<그림 2.3.5> TCP 통신 구조 ····23

<그림 2.4.1> ESP32 소프트웨어 흐름도 ····25

<그림 2.4.2> ADS-TWR 데이터 버퍼 내용변화 ····27

<그림 2.4.3> Tag가 Anchor A/B를 번갈아 폴링하는 전체 프로토콜 ····29

<그림 2.4.4> 데이터 후처리 과정 ····31

<그림 2.4.5> 앵커 A, 앵커 B 수집한 거리 데이터 ····32

<그림 2.4.6> 앵커 A, 앵커 B 거리보정 곡선 ····33

<그림 2.4.7> 카트 중심 좌표계 및 앵커 배치 모델 ····35

<그림 2.4.8> 추종 제어 구조 ····36

<그림 2.4.9> 내부 스레드 구조 및 데이터 흐름도 ····40

<그림 2.4.10> FSM 플로우 차트 ····43

<그림 2.5.1> 앵커 A 정적 실험 : Raw 데이터와 이동평균 필터 결과 비교 ····47

<그림 2.5.2> 앵커 B 정적 실험 : Raw 데이터와 이동평균 필터 결과 비교 ····47

<그림 2.5.3> 앵커 A 확대: 고빈도 잡음 제거 효과 확인 ····48

<그림 2.5.4> 앵커 B 확대: 고빈도 잡음 제거 효과 확인 ····48
<그림 2.5.5> 앵커 A 동적 실험: 원시값 vs 스파이크 필터 반응성 비교 ····49
<그림 2.5.6> 앵커 A 동적 실험 변화 구간 확대 ····49
<그림 2.5.7> 앵커 B 동적 실험: 원시값 vs 스파이크 필터 반응성 비교 ····49
<그림 2.5.8> 앵커 A 동적 실험 변화 구간 확대 ····50
<그림 2.5.9> 앵커 A 보정 전·후 거리 비교 및 선형성 개선 ····50
<그림 2.5.10> 앵커 B 보정 전·후 거리 비교 및 선형성 개선 ····51
<그림 2.5.11> 장애물 회피 및 추종 주행 동작 시퀀스 ····52
<그림 2.5.12> 정상 주행 상태 모니터링 (대기 및 추종) ····54
<그림 2.5.13> 장애물 감지 및 회피 상태 알림 ····54

국문 요약

본 논문은 대형 마트 및 물류 현장에서 사용자의 이동 편의성을 높이고 작업 효율을 증대시키기 위해, UWB(Ultra-Wideband) 통신 기술을 활용한 자율 추종 스마트 카트 시스템을 제안한다. 기존의 블루투스(BLE)나 비전 기반 시스템이 가지는 거리 오차 및 높은 연산 비용 문제를 해결하기 위해, 정밀 거리 측정이 가능한 Decawave DWM1000 모듈과 ESP32, Raspberry Pi 4를 결합하여 임베디드 제어 시스템을 구축하였다. 제안된 시스템은 ADS-TWR 기법을 통해 cm 단위의 거리 정밀도를 확보하였으며, 다중 경로 간섭 및 안테나 지연으로 인한 비선형 오차를 줄이기 위해 구간별 적응형 보정 알고리즘과 이동평균 및 스파이크 적응 필터를 적용하였다. 또한, 두 개의 UWB 앵커를 이용한 삼각측량법으로 사용자의 상대 위치를 실시간으로 산출하고, 상태 기반 제어(FSM) 알고리즘을 통해 안정적인 추종 주행과 초음파 센서 기반의 장애물 회피 기능을 구현하였다. 실험 결과, 거리 추정 오차(RMSE)를 보정 전 대비 약 70~90% 감소시켰으며, 안드로이드 앱과의 연동을 통해 실시간 모니터링 및 제어가 가능한 완성도 높은 시스템임을 검증하였다.

영문 요약

This paper proposes an autonomous smart cart system utilizing Ultra-Wideband (UWB) technology to enhance user convenience and operational efficiency in logistics and shopping environments. To address the limitations of existing Bluetooth (BLE) and vision-based systems, such as distance inaccuracies and high computational costs, an embedded control system was developed using Decawave DWM1000 modules, ESP32, and Raspberry Pi 4. The proposed system achieves centimeter-level positioning accuracy using the Asymmetric Double-Sided Two-Way Ranging (ADS-TWR) method. To mitigate non-linear errors caused by multi-path interference and antenna delay, an adaptive calibration algorithm combined with moving average and spike adaptation filters was implemented. The system calculates the user's relative position in real-time through triangulation using two UWB anchors and executes stable following movements using a Finite State Machine (FSM) control logic, which includes ultrasonic sensor-based obstacle avoidance. Experimental results demonstrate that the Root Mean Square Error (RMSE) of distance estimation was reduced by approximately 70–90% compared to uncalibrated data. Furthermore, the system's practical viability was validated through seamless integration with an Android application for real-time monitoring and control.

제 1 장. 서론

1.1. 개발 배경

최근 대형 마트 및 물류 산업 현장에서는 소비자의 이동 편의성을 향상시키고 작업 효율을 높이기 위한 자동화 기술 도입 요구가 지속해서 증가하고 있다. 특히 쇼핑 환경에서는 고령자, 장애인, 아동 동반 고객 등 무거운 카트를 직접 운용하기 어려운 사용자를 위한 주행 보조 시스템이 필수적이며, 물류 창고에서는 반복적인 중량물 운반으로 인한 작업자의 피로 누적과 안전사고 예방이 중요한 과제로 대두되고 있다. 이러한 문제를 해결하기 위한 방안으로 사용자를 인식하고 자율적으로 따라다니는 스마트 추종 카트(Smart Follow-cart)의 필요성이 강조되고 있다.

기존의 추종 시스템은 주로 저전력 블루투스(BLE)나 비전(Vision) 기술을 활용해 왔다. 그러나 블루투스 기반 시스템은 신호의 수신 강도인 RSSI(Received Signal Strength Indicator)를 이용해 거리를 측정하기 때문에 거리 오차가 크고, 주변 전파 간섭이나 인체에 의한 신호 차단 현상으로 인해 안정적인 추종이 어렵다는 한계가 있다. 또한, 카메라를 활용한 비전 기반 기술은 높은 인식 정확도를 제공하지만, 고성능 프로세서와 GPU 연산 자원이 필요하여 시스템 비용이 상승하며, 조명 변화, 장애물 가려짐, 복잡한 배경 등 환경적 요인에 민감하여 실내 마트와 같은 가변적인 환경에서 신뢰성을 확보하기 어렵다. 따라서 기존 기술의 한계를 극복하고, 합리적인 비용으로 실내에서 정밀하고 안정적인 주행이 가능한 새로운 거리 측정 및 제어 기술의 도입이 요구된다.

1.2. 개발 목표

본 연구에서는 비전 기반 방식 대비 구현 복잡도 및 연산 자원 요구가 낮고, BLE 기반 방식 대비 거리 추정 변동성 및 환경 간섭 영향이 상대적으로 작은 TOF(Time Of Flight) 기반 UWB(Ultra-Wideband) 거리 측정 기술을 핵심 요소로 채택하였다. UWB 통신은 매우 짧은 시간 폭의 펄스를 넓은 주파수 대역으로 송신하는 방식으로, 신호의 시간 해상도가 높아 거리 측정에 유리한 특성을 가진다. 특히 실내 환경에서 발생하는 다중 경로 반사 및 전파 간섭의 영향을 상대적으로 적게 받으며, 신호의 도달 시간을 정밀하게 측정할 수 있어 TOF 기반 거리 측정에 적합하다. 이러한 특성으로 인해 UWB 통신은 실내 위치 추정 및 사용자 추종 시스템에 널리 활용되고 있다.

따라서 UWB 기반 거리 데이터를 중심으로 사용자의 위치와 이동을 실시간으로 인식하고, 카트가 사용자를 자동으로 추종하며 장애물을 능동적으로 회피할 수 있는 자율 추종 주행 기능을 구현하는 것을 목표로 한다.

시스템은 사용자가 카트를 물리적으로 조작하지 않더라도 실내 환경에서 자연스럽게 따라

오도록 설계되며, 스마트폰 애플리케이션을 통해 ‘추종’, ‘대기’ 등의 동작 상태를 명령하고 주행 상태를 모니터링할 수 있는 통합 제어 환경을 제공한다. 카트는 사용자를 추종하는 과정에서 전방 장애물이 감지될 경우 자동으로 정지하거나 회피 동작을 수행하며, 사용자와 카트 간의 거리를 실시간으로 산출하여 스마트폰을 통해 시각적으로 제공한다. 또한 일정 거리 이내로 접근할 경우 카트가 자동으로 정지하도록 하여, 실내 환경에서 보다 안전하고 안정적인 사용자 추종이 가능하도록 한다.

이를 바탕으로 본 연구에서는 다음과 같은 세부 개발 목표를 설정한다.

(1) UWB 기반 실시간 거리 측정 및 신뢰성 향상 알고리즘 구현
(2) 좌·우 거리 정보를 이용한 사용자 방향 판단 기능 구현
(3) 거리 및 방향 정보를 기반으로 한 자동 사용자 추종 제어 기능 구현
(4) 주행 중 장애물 감지를 통한 자동 회피 및 안전 제어 기능 구현
(5) 스마트폰 애플리케이션을 이용한 주행 상태 제어 및 모니터링 기능 구현

다음 장에서는 제시한 개발 목표들을 구현하기 위한 전체 시스템의 하드웨어 및 소프트웨어 구성에 대해 상세히 설명한다.

제 2 장. 본론

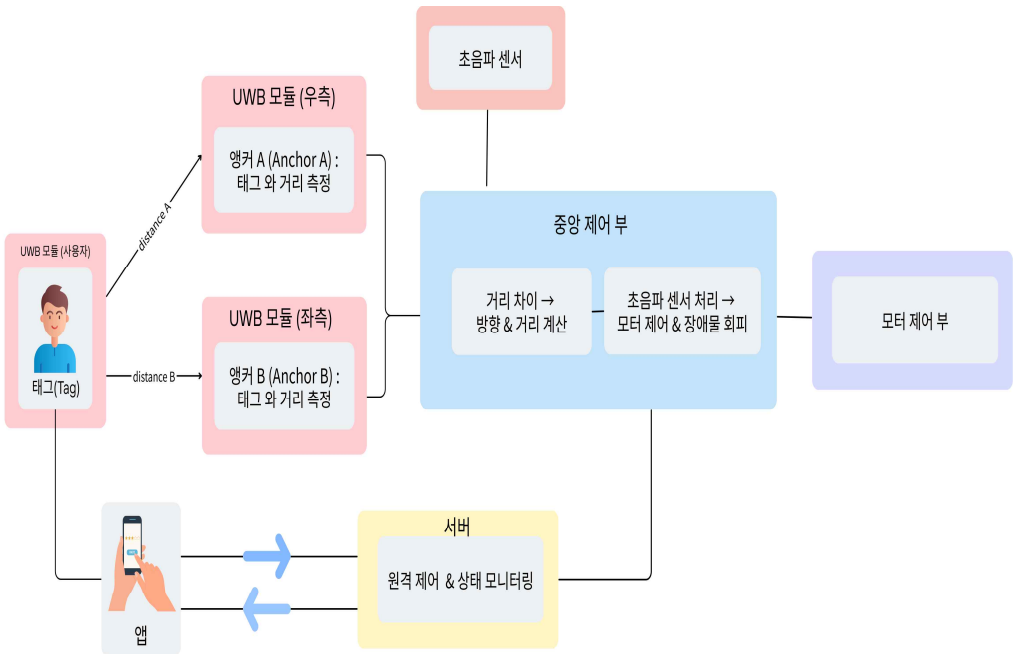
2.1. 전체 시스템 구성도

본 연구에서 구현한 스마트 카트 시스템은 UWB 기반 거리 측정 모듈, 중앙 제어부, 모터 구동부, 초음파 센서, 서버, 그리고 모바일 애플리케이션으로 구성된다. 전체 시스템 구성도는 <그림 2.1.1>에 나타내었다.

사용자는 UWB 기반 거리 측정 모듈을 소지하며, 이후 본 논문에서는 이를 태그(Tag)라 정의한다. 스마트 카트에는 동일한 구성의 UWB 기반 거리 측정 모듈이 좌·우 측에 각각 설치되어 있으며, 이를 각각 앵커 A(Anchor A)와 앵커 B(Anchor B)로 정의한다. 각 앵커는 태그와의 UWB 통신을 통해 실시간 거리 정보(distance A, distance B)를 측정한다.

각 앵커에서 측정된 거리 데이터는 중앙 제어부로 전송되며, 중앙 제어부는 수신된 거리 정보를 기반으로 사용자와 카트 간의 상대적인 방향 및 거리를 계산한다. 계산된 결과는 카트의 주행 방향 결정에 사용되며, 해당 주행 명령은 모터 구동부로 전달되어 카트가 사용자를 추종하는 동작을 수행한다.

또한 주행 중에는 초음파 센서로부터 입력되는 센서 데이터가 중앙 제어부에 전달되며, 해당 정보는 주행 제어에 반영되어 충돌을 방지하고 안전한 이동이 가능하도록 한다. 중앙 제어부는 계산된 거리 정보 및 시스템 상태 정보를 서버로 전송하며, 서버는 TCP/IP 기반 통신을 통해 모바일 애플리케이션과 연동된다. 사용자는 모바일 애플리케이션을 통해 카트의 주행 상태를 모니터링하고, 추종 및 대기와 같은 동작 명령을 원격으로 제어할 수 있다.



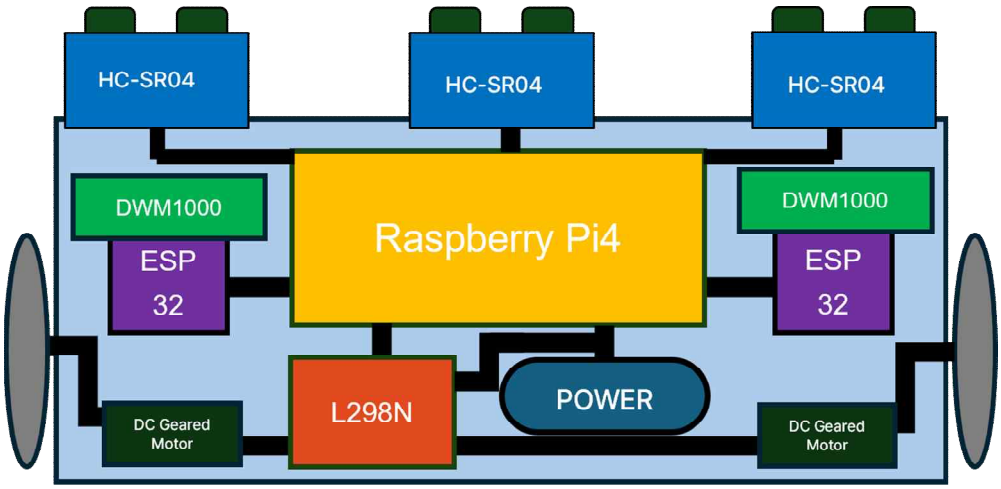
<그림 2.1.1> 시스템 구성도

2.2. 하드웨어 구성

본 절에서는 전체 시스템 중 스마트 카트 내부에 장착된 하드웨어 구성 요소를 중심으로, 각 구성 요소의 역할과 선택 이유를 설명한다. 카트 내부 하드웨어는 UWB 기반 거리 측정 모듈, 중앙 제어부, 모터 구동부, 그리고 장애물 감지를 위한 초음파 센서부로 구성된다. <그림 2.2.1>은 하드웨어가 장착된 스마트 카트의 실물 모습을 나타내며, <그림 2.2.2>는 본 연구에서 실제로 사용한 하드웨어 모듈을 기준으로, 카트 내부 하드웨어 구성 요소 간의 연결 구조를 나타낸다. 각 하드웨어 구성 요소는 중앙 제어부를 중심으로 상호 연동되도록 배치되었으며, 안정적인 주행과 센서 데이터 처리를 고려하여 카트 하부에 고정하였다.



<그림 2.2.1> 하드웨어 장착 카트 실물



<그림 2.2.2> 하드웨어 전체 구성도

(1) UWB 기반 위치 추정 모듈

본 연구에서는 UWB 기반 거리 측정 모듈로 Decawave사의 DWM1000을 채택하였다. DWM1000은 IEEE 802.15.4a 표준을 기반으로 한 UWB 통신 트랜시버로, 짧은 시간 폭의 고주파 펄스 (3.5–6.5 GHz)를 송수신하고 신호의 TOF을 이용하여 거리를 측정한다. 제조사 데이터시트에 따르면, DWM1000은 양방향 거리 측정(Two-Way Ranging) 기반 위치추정 시스템에서 약 ±10cm 수준의 거리 측정 정밀도를 제공할 수 있다. [1]

DWM1000 모듈의 제어 및 거리 데이터 처리를 위해 ESP32 마이크로컨트롤러를 사용하였다. DWM1000은 독립적인 MCU를 포함하지 않으므로, 통신 제어 및 데이터 처리를 위해 외부 마이크로컨트롤러가 필요하다. ESP32는 3.3V 로직 레벨을 사용하여 DWM1000과의 전기적 호환성이 우수하며, SPI 인터페이스를 통해 안정적인 데이터 송수신이 가능하다. 또한 충분한 연산 자원과 다양한 통신 인터페이스를 제공하여, UWB 거리 측정 제어와 중앙 제어부와의 데이터 연동을 동시에 수행하기에 적합하다.

이와 같은 이유로 본 연구에서는 DWM1000과 ESP 32를 결합하여 하나의 UWB 기반 위치추정 모듈로 구성하였으며, 해당 모듈을 태그, 카트 좌·우 측에 설치된 앵커 A, 앵커 B로 활용한다. <그림 2.2.3>은 ESP32와 DWM1000을 결합하여 구성한 UWB 기반 위치추정 모듈의 실물 사진을 나타낸다.



<그림 2.2.3> (A) ESP32 (B) DWM1000

(2) 중앙처리 제어부

본 연구에서는 스마트 카트의 중앙 제어부로 라즈베리 파이(Raspberry Pi 4 Model B)를 사용하였다. 라즈베리 파이는 다중 USB 인터페이스를 제공하여, 카트 좌·우에 설치된 두 개의 UWB 앵커로부터 USB 직렬 통신을 통해 거리 데이터를 안정적으로 수신할 수 있다. 또한 충분한 연산 성능을 제공하여, 수신된 거리 데이터를 실시간으로 처리하고 사용자와 카트 간의 상대적인 방향 및 거리 차이를 계산하는 중앙 제어 장치로 활용하기에 적합하다.

라즈베리 파이는 PWM(Pulse Width Modulation) 신호를 생성하여 모터 구동부에 전달함으로써 카트의 속도 제어 및 방향 전환을 수행한다. 또한 리눅스 기반의 운영체제를 지원하여 TCP/IP 기반 네트워크 통신 환경을 손쉽게 구성할 수 있으며, 이를 통해 원격 서버 및 모바

일 애플리케이션과 실시간 데이터 송수신이 가능하다. 이러한 통신 구조를 통해 주행 상태와 거리 정보를 사용자에게 제공할 수 있다.

이와 같은 특성으로 인해 라즈베리 파이는 본 연구에서 요구되는 실시간 데이터 처리, 다중 센서 연동, 네트워크 통신 기능을 통합적으로 수행할 수 있는 중앙 제어부로 판단되어 채택하였다. <그림 2.2.4>는 사용한 라즈베리 파이 모델인 Raspberry Pi 4 Model B의 사진이다.



<그림 2.2.4> Raspberry Pi 4 Model B

(3) 모터 구동부

스마트 카트의 주행을 담당하는 핵심 부품은 DC 모터이다. 스마트 카트는 정지 상태에서 사용자를 따라 즉시 가속해야 하며, 동시에 실내 환경에서도 안전하게 제어되어야 한다. 따라서 구동 모터에는 충분한 정격 토크와 세밀한 속도 제어 능력, 낮은 전력 소비가 요구된다. <표 2.1> 은 후보로 검토된 모터의 주요 사양을 나타낸다.

<표 2.1> 모터 주요 사항

구분	모델명	정격전압	무부하 속도 (RPM)	정격 토크 (kg.cm)	소비전류	비고
A형	TT DC Geared Motor	9V	150	4.5	0.6A	저가형, 마찰 손실 큼
B형	37GB555 1000RPM	12V	100	10	0.9A	고토크, 무게 무거움
C형	JGA25-370 100RPM	12V	100	2.5	0.35A	소형, 효율 우수
D형	775 DC Motor	12V	6000	0.7	2.5A	고속형, 소음 많음

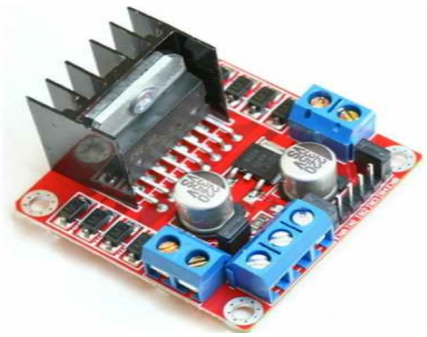
<표 2.1>과 같이, 4종의 모터를 성능과 제어 특성 측면에서 비교 결과 TT 모터는 가격이 저렴하지만, 하우징 내 마찰 손실이 커서 장시간 주행 시 효율이 급격히 저하되었다. 37GB555 모터는 높은 토크를 제공하지만, 모터 자체의 무게가 약 350g으로 무겁고, 카트 프레임의 하중 분포를 고려했을 때 적합하지 않았다. 775 모터는 고속 회전에 유리하지만, 감속기가 없고 전류 소모가 커 실내 자율주행 플랫폼에는 부적합하였다. 반면 JGA25-370은 소형이면서도 충분한 구동 토크를 제공하고, 좌·우에 두 개의 모터를 적용하면 전체 하중

약 4~5kg의 카트를 안정적으로 구동할 수 있는 성능을 갖춘 것으로 판단되었다. 따라서 JGA25-370 모터를 채택하였다. <그림 2.2.5>는 채택한 모터인 JGA25-370 모터의 사진이다.



<그림 2.2.5> JGA25-370 모터

각 모터는 카트의 좌우 바퀴에 연결되어 있으며, 두 바퀴의 회전 속도 차이를 이용해 전진, 후진, 회전 등 다양한 주행 패턴을 구현할 수 있다. 모터의 제어는 L298N 모터 드라이버를 통해 이루어진다. 해당 드라이버는 최대 2A의 출력 전류를 지원하며, 두 개의 모터를 독립적으로 제어할 수 있어 카트의 좌우 구동에 적합하다. 라즈베리 파이에서 생성된 PWM 신호를 L298N 모터 드라이버 입력단으로 전달하여 속도와 방향을 제어하며, 두 바퀴의 회전 속도를 비율적으로 조정하여 직진, 회전, 후진 등의 동작을 구현한다. 또한, 과전류 보호 회로가 내장되어 있어 장시간 주행 시 모터 과열이나 드라이버 손상 위험이 적다. 이러한 구조는 추종 주행 시 사용자 거리 변화에 따른 속도 보정 제어를 안정적으로 수행할 수 있도록 하며, 카트의 동작 응답성과 에너지 효율을 동시에 확보하였다. [2] <그림 2.2.6>은 L298N 모터 드라이버의 사진이다.



<그림 2.2.6> L298N 모터 드라이버

(4) 초음파 센서부

주행 중 장애물을 감지하고 충돌을 방지하기 위해 초음파 센서를 적용하였다. 사용된 센서는 HC-SR04 모델로, 저전력·저비용이면서도 비교적 안정적인 거리 인식이 가능하여 모바일 로봇 및 자율 주행 플랫폼에서 널리 사용되는 대표적인 초음파 거리 센서이다. HC-SR04는 초음파의 반사 원리를 이용하여 물체까지의 거리를 계산한다. 센서 전면에는 송신기와 수신기가 한 쌍으로 배치되어 있으며, 송신기에서 40kHz 주파수의 초음파 펄스를 방출하고, 이 음파가 물체에 반사되어 돌아오는 시간을 측정하고 <식 2.1>로부터 거리를 산출한다. [3]

$$d = \frac{v \times t}{2}$$ <식 2.1>

여기서 v는 음속(약 343m/s), t는 초음파가 왕복하는 시간이다. HC-SR04는 내부 타이머 회로를 이용해 Trigger 신호가 입력되면 8회의 40kHz 펄스를 송출하고, 반사 신호를 수신하는 동안 ECHO 핀을 HIGH 상태로 유지한다. 이 ECHO 신호의 지속 시간을 측정하면 거리 계산이 가능하다. 센서의 동작 전압은 5V, 동작 전류는 약 15mA로 비교적 낮아 마이크로컨트롤러와의 연동이 쉽다. 다만 라즈베리 파이의 GPIO 입력 핀은 3.3 V 로직 레벨을 사용하므로, ECHO 신호 입력 시 전압 레벨 차이에 따른 손상을 방지하기 위해 전압 분배 회로를 적용하여 5V 신호를 3.3V로 변환하여 연결하였다.

제조사 데이터시트에 의하면 HC-SR04의 실효 측정 거리는 2cm~400cm이며, 이 구간 내에서 약 ±0.3cm 수준의 정확도를 제공한다. [4] 탐지 각도는 약 30° 로, 전방의 넓은 영역을 커버할 수 있다. 스마트 카트에서는 이 센서를 전방 중앙부에 3개 장착하여 주행 중 전방 장애물을 실시간으로 감지하도록 구성하였다. <그림 2.2.7>은 초음파 센서 HC-SR04의 사진이다.



<그림 2.2.7> 초음파 센서 HC-SR04

(5) 전원 공급부

스마트 카트 시스템은 구동 모터, 제어부, 센서 모듈 등 다양한 하드웨어가 동시에 동작하기 때문에 안정적이고 효율적인 전원 공급이 필수적이다. 본 연구에서는 모터 구동부의 주 전원으로 12V 리튬이온 배터리 팩(3셀, 2,200mAh)을 사용하였고 전체 시스템의 주 전원은 5V 3A 리튬이온 보조 배터리를 사용하였다. <그림 2.2.8>은 모터 구동부에 사용된 12V 리튬이온 배터리 팩의 사진이다.



<그림 2.2.8> 12V 리튬이온 배터리 팩

2.3. 통신 시스템

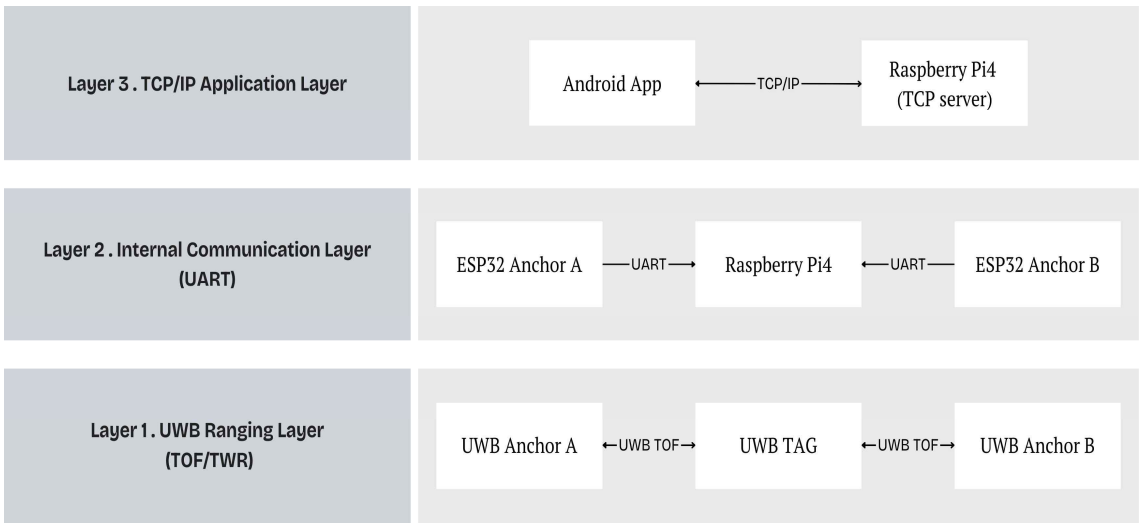
본 스마트 카트 시스템의 통신 구조는 UWB 기반 거리 측정 통신, UART 기반 내부 장치 간 통신, 그리고 TCP/IP 기반 외부 통신으로 구성된 3계층 통신 구조로 이루어진다. 이러한 구조를 통해 센서 계층에서 생성된 거리 정보가 중앙 제어부와 사용자 인터페이스까지 효율적으로 전달되도록 설계하였다.

먼저, 태그와 카트에 설치된 앵커 A 및 앵커 B 간의 거리 측정은 UWB 통신 기반의 양방향 거리 측정 방식을 통해 수행된다. 태그는 두 개의 앵커와 교대로 거리 측정을 수행하도록 구성되어 있으며, 이를 통해 약 20Hz의 갱신 주기로 새로운 거리 데이터가 생성된다.

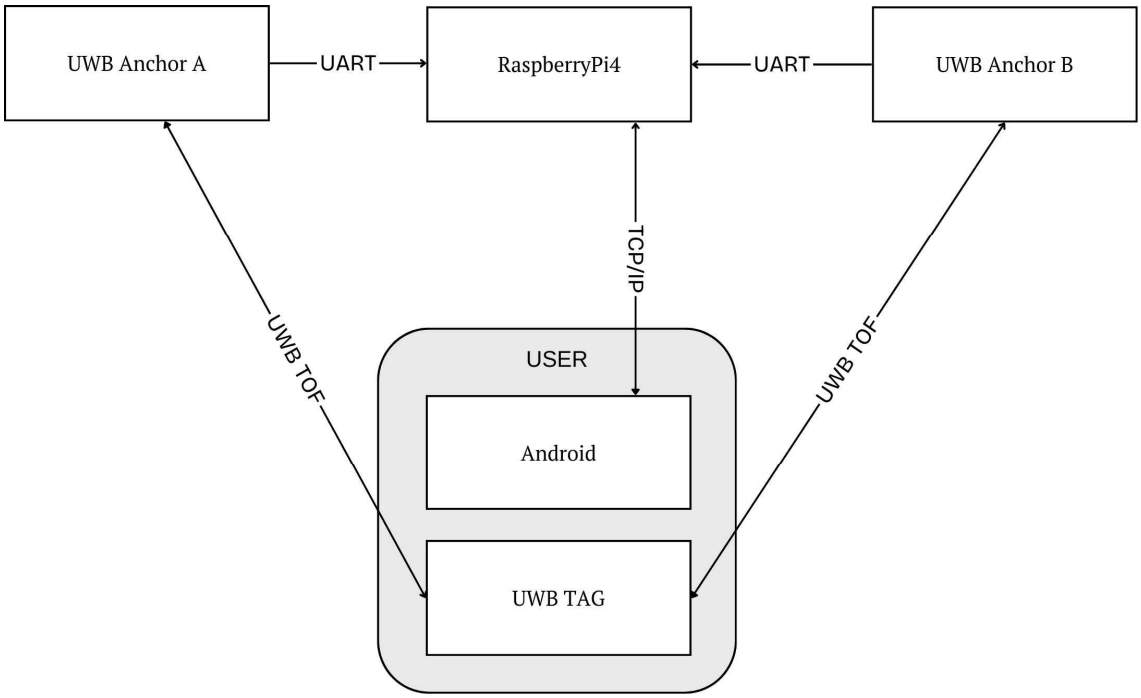
각 앵커에서 측정된 거릿값은 UART 통신(115200 bps)을 통해 중앙 제어부인 라즈베리 파이로 전달된다. 라즈베리 파이는 수신된 거리 데이터를 기반으로 사용자 위치추정 및 주행 제어 알고리즘을 수행하며, 거리 정보는 같은 주기로 상위 제어 모듈에 반영된다.

또한 중앙 제어부는 TCP/IP 기반 네트워크 통신을 통해 스마트폰 애플리케이션과 연동된다. 생성된 거리 데이터와 카트의 운용 상태 정보는 실시간으로 스마트폰 애플리케이션에 전송되며, 사용자는 애플리케이션을 통해 현재 거리(DIST)와 동작 상태(MODE)를 확인할 수 있다. 또한 START, STOP, RESUME 명령을 전송하여 카트의 동작을 원격으로 제어할 수 있다.

<그림 2.3.1>은 본 시스템의 3계층 통신 구조를 나타내며, <그림 2.3.2>는 전체 시스템의 통신 흐름을 개략적으로 나타낸다.



<그림 2.3.1> 3계층 통신 구조



<그림 2.3.2> 전체 시스템 통신 구조

(1) UWB 거리 측정 통신 구조

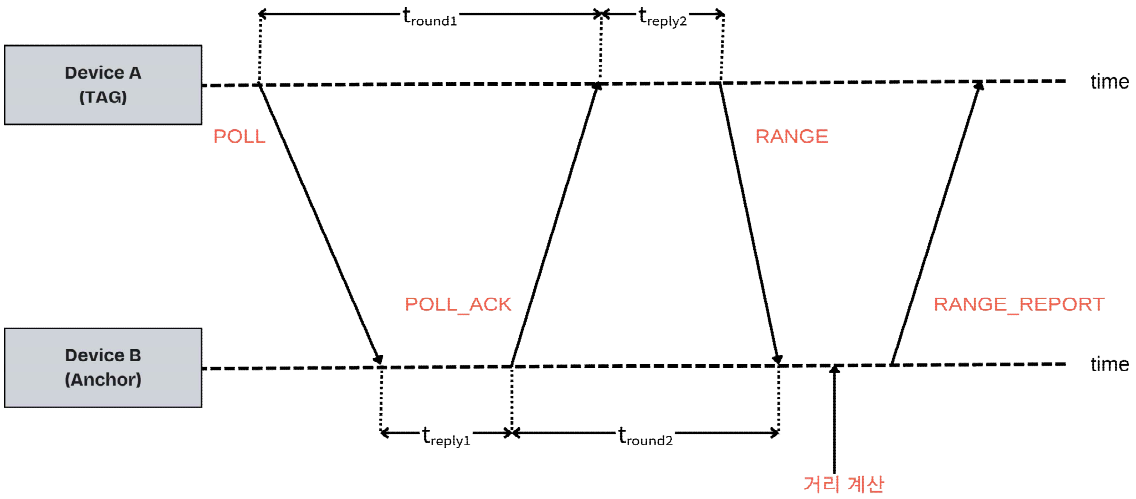
UWB 기반 거리 측정은 신호가 송신 단말에서 수신 단말까지 도달하는 데 걸리는 시간, 즉 신호의 비행시간 TOF를 측정하여 거리를 계산하는 방식이다. 전파의 전파 속도가 일정하다는 점을 이용하여, TOF에 전파 속도를 곱함으로써 두 단말 간의 거리를 산출할 수 있다.

1) Asymmetric Double-Sided Two-Way Ranging

DWM1000은 정밀한 거리 측정을 위해 ADS-TWR(Asymmetric Double-Sided Two-Way Ranging) 방식을 사용한다. ADS-TWR는 양 단말(Device A, Device B)이 서로 송신 및 응답 메시지를 주고받으며, 각 구간의 왕복 시간(Round-trip Time)과 응답 지연 시간(Reply Delay Time)을 조합하여 최종 TOF를 계산하는 방식이다. 이 방식은 단말 간 클록 불일치 문제를 해결하도록 설계되었으며, 두 장치의 내부 클록 속도가 미세하게 다르더라도 이를 수식적으로 보정하여 높은 정확도와 안정성을 제공한다. ADS-TWR의 메시지 교환 과정은 <그림 2.3.3>과 같다. [5]

여기서 사용되는 시간 변수는 다음과 같이 정의된다.

- t_{round1} : Device A 기준 첫 번째 왕복 시간
- t_{round2} : Device B 기준 두 번째 왕복 시간
- t_{reply1} : Device B의 첫 번째 응답 지연 시간
- t_{reply2} : Device A의 두 번째 응답 지연 시간



<그림 2.3.3> ADS-TWR 통신

구체적인 동작 과정은 다음과 같다. 먼저 태그(Device A)는 앵커(Device B)에게 POLL 프레임을 전송하고, 앵커는 이를 수신한 후 POLL_ACK(Response)로 응답한다. 이후 태그는 첫 번째 왕복 구간의 타임 스탬프 정보를 포함한 RANGE 메시지를 전송하며, 앵커는 최종적으로 거리를 계산하고 태그에게 RANGE_REPORT를 전송한다. 이 과정에서 획득한 4개의 시간 변수 t_{round1} , t_{round2}, t_{reply1}, t_{reply2} 을 이용하여 ToF는 <식 2.3.1>과 같이 계산된다.

$$TOF = \frac{(t_{round1} \cdot t_{round2}) - (t_{reply1} \cdot t_{reply2})}{t_{round1} + t_{reply1} + t_{round2} + t_{reply2}} \qquad \text{<식 2.3.1>}$$

계산된 TOF를 이용하여 두 장치 간의 물리적 거리 d는 다음과 같이 구할 수 있다.

$$d = TOF \cdot c$$

여기서 c 는 빛의 속도이며 약 $c = 3 \times 10^8$ m/s 이다.

본 연구에서는 ADS-TWR 거리 측정 과정을 DWM1000 모듈과 SPI 방식으로 연결된 ESP32 마이크로컨트롤러의 펌웨어 에서 수행하도록 구성하였다. ESP32 내부 펌웨어에는 오픈소스 기반의 DW1000Ng 라이브러리를 적용하여 DWM1000 하드웨어를 제어하였다. 해당 라이브러리는 POLL, POLL_ACK, RANGE 메시지 생성, 고정밀 타임스탬프 읽기, 송수신 인터럽트 처리 등 TWR 기반 거리 측정의 전 과정을 효율적으로 처리한다. 또한 DW1000 칩셋의 복잡한 레지스터 설정, UWB 통신 파라미터 설정, 안테나 지연(Antenna Delay) 보정과 같은 핵심 기능들을 추상화하여 제공함으로써, ESP32가 안정적으로 UWB 기반 Ranging 프로토콜을 수행할 수 있도록 지원한다. 단일 앵커와 태그간의 1대1 ADS-TWR 은 약 10ms 미만의 시간이 걸린다.

2) 태그 - 앵커 간 교대로 수행되는 양방향 통신 구조

본 시스템은 두 개의 앵커(Anchor A, Anchor B)를 사용하여 사용자로부터 두 개의 거리변수를 안정적으로 확보한다. 태그는 다음 순서를 반복한다.

1. Anchor A와 ADS-TWR 수행 → 거리 d_A 획득
2. Anchor B와 ADS-TWR 수행 → 거리 d_B 획득

각각의 ADS-TWR 교환은 약 10ms 이내의 간격으로 이루어지며, 두 교환 사이의 delay를 거쳐 거리 정보 (d_A , d_B) 가 라즈베리 파이에 도달한다. 이렇게 얻은 두 거리 값은 소프트웨어 파트에서 설명할 사용자 위치 연산에 활용된다,

3) 환경 보정 및 거리 캘리브레이션

DWM1000은 매우 정밀한 시간 스탬프를 제공하지만, 실제 환경에서는 다음 요인으로 인해 비선형 오차가 발생한다.

- 안테나 지연 (Antenna Delay)
- 신체・카트・지면에 의한 NLOS(Non-Line-of-Sight)
- 반사파(Multipath reflection)
- 앵커 간 설치 각도 및 높이 차이

데이터시트에는 Antenna Delay Calibration 항목이 존재하며, Decawave는 실제 거리에서 측정한 값을 기준으로 안테나 딜레이 값을 설정하는 것을 권장한다. 본 연구에서도 같은 방법을 사용하여 실측 거리 3m를 기준으로 안테나 딜레이 값 A: 16530, B: 16530, 태그: 16436을 설정하였다. 안테나 딜레이 값 설정에 추가로 3차 다항식 기반의 보정 식을 적용하여 오차를 기존 대비 많이 감소시켰다.

(2) Raspberry Pi 내부 통신 (UART)

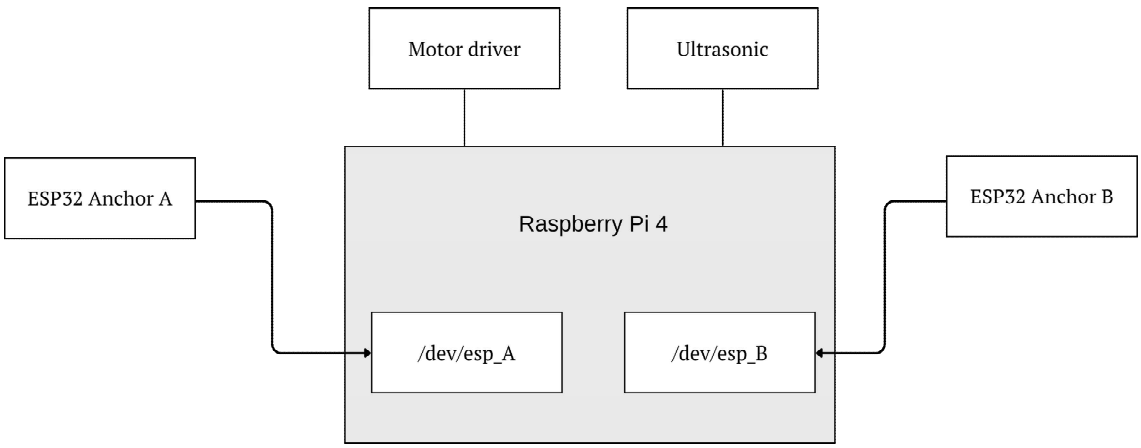
본 시스템에서는 라즈베리 파이와 두 개의 ESP32 앵커 간의 거리 데이터 전송을 위해 직렬 비동기 통신 방식인 UART(Universal Asynchronous Receiver/Transmitter)를 사용하였다. 라즈베리 파이는 USB 인터페이스를 통해 두 개의 ESP32 모듈을 개별 장치로 인식하며, udev 규칙을 적용하여 포트 연결 순서와 무관하게 각각 /dev/esp_A, /dev/esp_B라는 고정된 링크로 대응하여 장치 인식의 모호성을 제거하였다. 두 개의 ESP32를 단일 포트가 아닌 독립된 UART 포트로 병렬 연결한 주된 이유는 시간 동기성 확보와 실시간 병렬 처리 때문이다. 두 앵커는 비동기적으로 ADS-TWR을 수행하므로 거리 데이터 생성 시점이 서로 다르다. 따라서 각 채널을 독립적인 입력으로 분리하여 데이터 수신 지연과 병목 현상을 최소화

하였다.
비동기 통신 환경에서 데이터 프레임의 경계를 명확히 구분하고 파싱 신뢰성을 높이기 위해, 본 연구에서는 다음과 같은 고정 길이 문자열 포맷을 정의하여 사용하였다.

Format : “ R: <Distance> @ <Timestamp> ”

예를 들어, “R: 1.234 @ 1715793840.123“과 같은 형태의 패킷이 전송된다. 여기서 R은 거리(Range) 데이터임을 나타내는 식별자이며, @ 문자를 구분자로 사용하여 거리 값과 측정 시각(Timestamp)을 분리한다. 라즈베리 파이의 제어 소프트웨어는 각 UART 포트로부터 수신된 문자열을 파싱하여, 이를 각각 A 거리 값과 B 거리 값 변수에 저장하고 필터링을 걸쳐 상위 제어 알고리즘의 입력으로 사용한다.

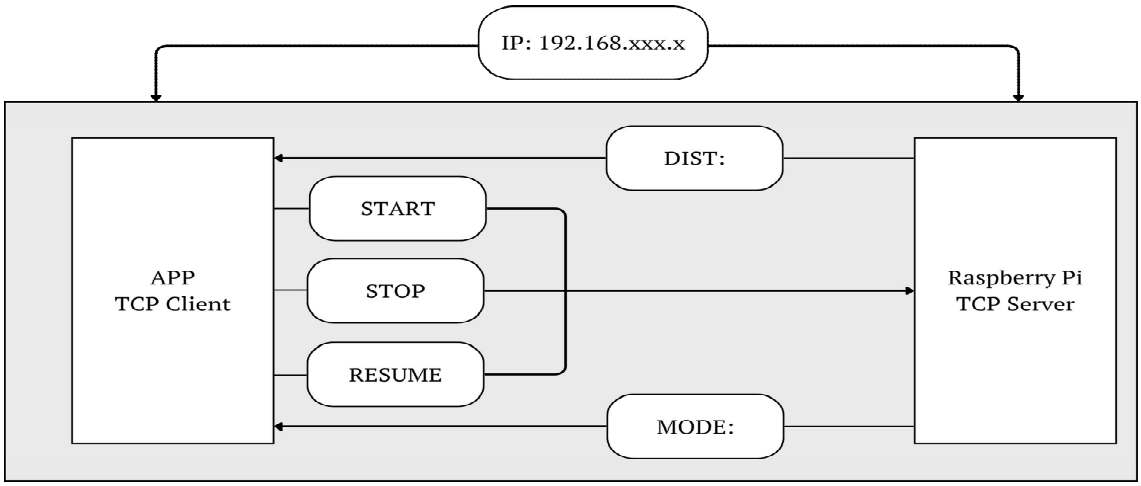
라즈베리 파이는 시스템의 중앙 통신 허브 역할을 수행하며, UWB 데이터뿐만 아니라 모터 드라이버 제어를 위한 PWM 신호 생성, 초음파 센서 데이터 수신 등 다양한 주변기기와의 입출력을 동시에 처리한다. UART 기반의 분산 처리 구조는 이러한 멀티태스킹 환경에서 연산 부하를 분산시키고 제어 주기의 실시간성을 보장하는 핵심적인 역할을 한다. 라즈베리 파이와 주변기기의 전체 연결 도면은 <그림 2.3.4>와 같다.



<그림 2.3.4> 라즈베리파이 내부 연결 장치

(3) Raspberry Pi와 스마트폰 간 통신 (TCP/IP)

라즈베리 파이는 UWB 거리 정보 및 센서 데이터를 종합하여 실시간으로 사용자 인터페이스(스마트폰 앱)와 데이터를 공유해야 한다. 이를 위해 라즈베리 파이를 TCP 서버(TCP Server)로, 스마트폰 앱(Android)을 TCP 클라이언트(TCP Client)로 구성하는 양방향 소켓 통신 구조를 사용하였다. <그림 2.3.5>는 TCP 통신 구조를 보여준다.



<그림 2.3.5> TCP 통신 구조

1) 통신 프로토콜 및 데이터 포맷

라즈베리 파이와 스마트폰 앱 간의 통신은 하드웨어 제어와는 독립된 스레드에서 비동기 방식으로 동작하며, 고정된 포맷의 문자열을 통해 데이터를 교환한다. 라즈베리 파이는 고정 포트 번호(5000)로 서버를 생성하며, 스마트폰 앱은 해당 서버에 접속하여 소켓 연결을 유지한다.

■ Raspberry Pi -> 스마트폰 (데이터 전송)

라즈베리 파이에서 스마트폰 앱으로 전송하는 거리 및 상태 정보는 스마트폰 앱에서의 파싱 신뢰성을 확보하기 위해 다음의 고정된 문자열 형식을 사용한다.

- 거리 정보 : DIST: x.xx (예: DIST:1.25)
- 상태 정보: MODE: STATE (예: MODE:TRACKING, MODE:ERROR)

■ Raspberry Pi -> 스마트폰 (데이터 전송)

스마트폰 앱은 사용자의 입력에 따라 다음 세 가지의 간결한 제어 명령을 라즈베리 파이 서버로 전송한다.

- START : 자동 추적 기능 활성화
- STOP : 카트 정지 및 대기 상태 진입
- RESUME : 안전 대기 상태에서 정상 추종 상태로 복귀

라즈베리 파이에서는 네트워크 메시지 수신/송신을 위한 스레드와 모터 제어를 위한 스레드를 독립적으로 분리하여, 통신 지연이 차량의 제어 주기에 영향을 주지 않도록 하였다. 앱

내부에서는 MyTcpService라는 백그라운드 서비스가 연결 유지 및 소켓 이벤트를 담당하며, 연결 끊김이 발생 시 자동으로 재연결을 시도하여 시스템의 연속적인 운영을 보장한다. 또한 통신 안정성을 확보하고 지연시간을 최소화하기 위해 라즈베리 파이는 로컬 Wi-Fi 네트워크 내에서 고정 IP(Static IP)를 사용하도록 구성하였다. 이는 실험 환경 및 제어 환경에 최적화된 저지연 통신을 가능하게 한다.

2.4. 소프트웨어 및 알고리즘

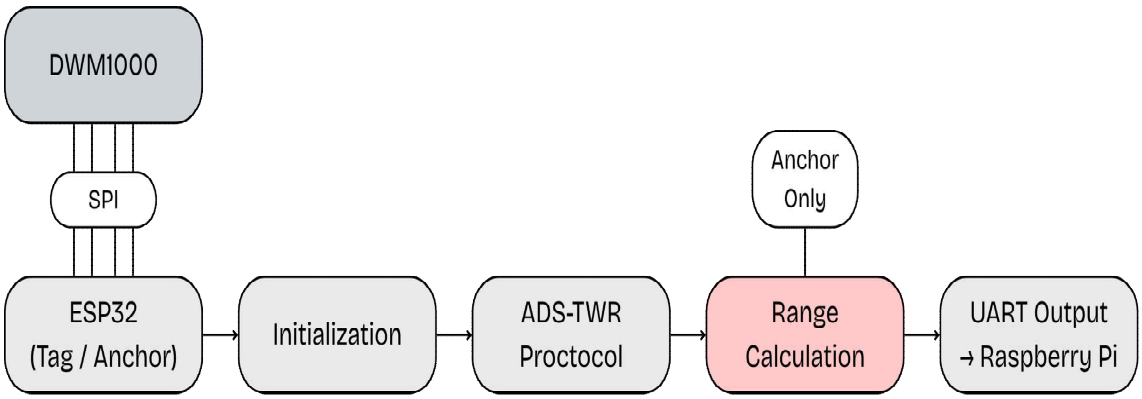
본 장은 스마트 카트 시스템의 핵심 기능인 거리 계산, 카트 이동 제어, 시스템 통합을 구현하는 소프트웨어 구조와 알고리즘을 기술한다. 앞서 설명된 통신 파트를 통해 ESP32에서 시작하여 라즈베리 파이로 전달되는 UWB 데이터가 카트 제어 값으로 변환되는 일련의 소프트웨어 과정을 제시한다. 본 연구의 중앙처리부는 라즈베리 파이 기반으로 구성되었으며, 데이터 수집, 후처리, 삼각측량(Trilateration), 추종 제어 등 대부분의 연산은 Python으로 구현되었다. 라즈베리 파이는 리눅스 기반의 임베디드 플랫폼이지만, 본 시스템의 연산 복잡도(50–100 Hz 수준 처리)가 C/C++을 요구할 정도의 엄격한 실시간 제약이 크지 않음을 고려하여, Python 기반 처리 모델을 채택하였다. 이는 Python의 직관적인 프로그래밍 구조와 풍부한 라이브러리 지원(pySerial, NumPy 등)을 활용하여 개발 속도를 높이고 유지보수 측면의 이점을 확보하기 위함이다. 전체 시스템 동작을 위한 소프트웨어 요소들은 다음과 같다

- ESP32 -
- 거리 데이터 수집: 태그로부터 앵커A, 앵커 B가 거리데이터 수집.

-라즈베리 파이 -
- 거리 데이터 필터링: ESP32로부터 거리 데이터를 전달 받아서 필터링.
- 사용자 거리 계산 및 방향 처리: 필터링된 데이터를 활용하여 사용자 거리 계산 및 운전
- 센서 데이터 처리: 카트 전방의 장애물 감지(초음파 센서) 및 안전 제어.
- 사용자 상호작용: 스마트폰 앱의 실시간 명령 처리(TCP/IP).
- 상태 관리: 상태 기반 제어(Finite State Machine)를 통한 전체 시스템 동작 관리.

(1) ESP32 Tag/Anchor 펌웨어 구조

본 시스템의 UWB 거리 측정은 ESP32와 DWM1000 모듈로 구성된 태그와 두 개의 앵커(Anchor A, Anchor B)에 의해 수행된다. 세 장치는 동일한 하드웨어 구조를 사용하지만, 동작 역할(이니시에이터/리스폰더)에 따라 펌웨어 로직이 구분된다. UWB 프로토콜 구현은 DW1000Ng 라이브러리를 기반으로 하였다. 전체 ESP32 소프트웨어 흐름은 <그림 2.4.1> 과 같다



<그림 2.4.1> ESP32 소프트웨어 흐름도

ESP32 소프트웨어 흐름은 4단계로 구성되어 있다.
① 초기 설정, ② ADS-TWR 프로토콜 처리, ③ 거리 계산, ④ 라즈베리 파이로 결과 전송.

1) 초기 설정

ESP32 부팅 후, 안정적인 UWB 통신 환경을 구축하기 위해 다음 항목들을 순차적으로 설정한다.

DWM1000 모듈 초기화 :

- SPI 인터페이스를 통해 DWM1000 모듈을 인식하고 통신을 초기화한다.
- SPI 핀 구성: MISO, MOSI, SCK, SS 핀을 특정 GPIO에 맵핑하여 물리적 통신 경로를 확립한다.

UWB 무선 파라미터 설정 :

- DW1000Ng 라이브러리의 API를 사용하여 UWB 통신 채널과 전송 속도를 정의한다.
- 채널: 6489.6 MHz (채널 5)
- PRF (Pulse Repetition Frequency): 16 MHz
- Preamble Length: 256

인터럽트 및 핸들러 등록 :

- 비동기 처리를 위해 수신 성공(handleReceived()), 수신 시간 초과(handleTimeout()), 수신 오류(handleError()) 발생 시 각각의 이벤트를 처리할 인터럽트 핸들러(ISR)를 등록한다.

장치 역할 및 주소 설정 :

- 태그, 앵커 A, 앵커 B의 역할을 정의하고, 라이브러리 함수를 사용하여 각 앵커에 고유한 MAC 주소(setDeviceAddress(1), setDeviceAddress(2))를 지정한다.
- 태그(Initiator)가 이 고유 주소를 기반으로 각 앵커와의 통신을 번갈아가며 수행하도록 로직을 구성하여 1대 2 통신 환경을 구현하였다.

2) ADS-TWR 프로토콜 처리 및 거리 계산

DW1000Ng 라이브러리와 인터럽트 기반 송수신 구조를 기반으로 하여, ADS-TWR를 실제로 구현한다. 태그는 앵커 A와 앵커 B와 순차적으로 ADS-TWR을 시도하여 두 개의 독립적인 거리(d_A, d_B)를 획득한다.

<ADS-TWR 프로토콜 과정 1>. 인터럽트 기반 송수신 구조

- 인터럽트 핸들러 :

```
C++

void handleSent() { sentAck =true; }            //수신했을 때 인터럽트
void handleReceived() { receivedAck =true; }  // 송신했을 때 인터럽트
```

- 인터럽트 내부에서는 오직 플래그(sentAck, receivedAck)만 설정하고, 실제 모든 프로토콜 진행과 시간 계산은 loop() 함수에서 플래그 상태를 확인하며 순차적으로 이루어진다. 이는 인터럽트 처리 시간을 최소화하여 시스템의 실시간 제약을 준수하고, 메인 루프에서 프로토콜 흐름을 제어함으로써 안정성을 확보하기 위함이다

<ADS-TWR 프로토콜 과정 2>. ADS-TWR 타임스탬프 확보 및 데이터 구조

<표 2.4.1> ADS-TWR 타임스탬프 기록 표

구분	변수명	의미	기록 시점
Tag	timePollSent	POLL 송신 시각	POLL 송신 직후
Tag	timePollAckReceived	POLL_ACK 수신 시각	POLL_ACK 수신 완료
Tag	timeRangeSent	RANGE 송신 시각	RANGE 송신 시각
Anchor	timePollRecieved	POLL 수신 시각	POLL 수신 완료 시
Anchor	timePollAckSent	POLL_ACK 송신 시각	POLL_ACK 송신 직후
Anchor	timeRangeReceived	RANGE 수신 시각	RANGE 수신 완료 시

한 쌍의 거리 계산에는 태그와 앵커가 교환하는 프레임 내에서 총 6개의 UWB 타임스탬프가 필요하며, 이를 기반으로 ADS-TWR 공식에 따른 비대칭 왕복 시간 계산이 이루어진다. <표 2.4.1> 은 코드에서 기록되는 타임스탬프의 변수명과 기록 시점이다.
태그에서 POLL을 송신한 직후 타임스탬프를 기록하지만 변수에 넣는 순간은 POLL_ACK를 수신완료한 순간이다. DW1000Ng::getTransmitTimestamp() 함수를 통해 POLL을 송신한 시각을 기억하고 있다가, POLL_ACK응답이 오면 변수에 값을 넣는다. 앵커는 총 6개의 타임스탬프를 조합하여 정확한 TOF를 계산한다.

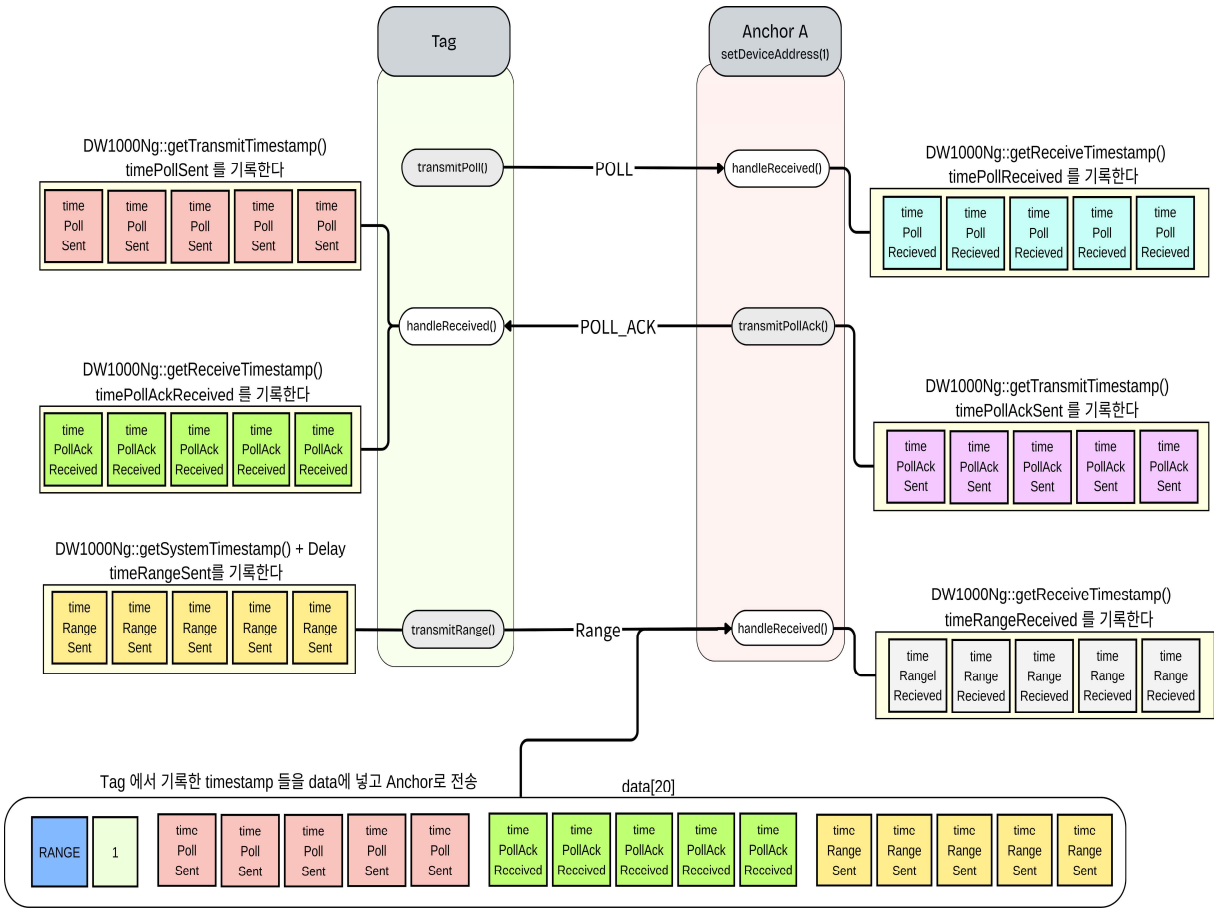
- **데이터 프레임 구조**

<표 2.4.2> 데이터 버퍼 구조

바이트 위치	크기	내용
data[0]	1Byte	메시지 타입(POLL, POLL_ACK, RANGE, RANGE_REPORT)
data[1]	1Byte	대상 앵커 주소 (1 또는 2)
data[2~6]	5Byte	timePollSent 값
data[7~11]	5Byte	timePollAckReceived 값
data[12~16]	5Byte	timeRangeSent 값
(나머지 3바이트는 여유분)		

태그와 앵커 간 메시지 교환에는 최대 20 바이트 크기의 데이터 버퍼(data[])가 사용된다. 특히 태그는 앵커에게 필요한 타임스탬프 3개를 전송해야 하므로, 데이터 구조는 <표 2.4.2>와 같이 구성하였다. 각 타임스탬프는 DW1000의 40-bit (5바이트) 시간 값이며,

DW1000NgUtils::writeValueToBytes() 및 DW1000NgUtils::bytesAsValue() 함수를 이용해 바이트 단위로 변환 및 복원된다. <그림 2.4.2>는 ADS-TWR 메시지 교환 중 타임스탬프 기록 시점과 데이터 버퍼 저장 과정을 시각적으로 나타낸 것이다.



<그림 2.4.2> ADS-TWR 데이터 버퍼 내용변화

<ADS-TWR 프로토콜 과정 3>. Tag 및 Anchor의 프로토콜 처리 흐름

프로토콜의 복잡도는 태그와 앵커의 역할에 따라 구분되며,
loop() 함수 내에서 sentAck/receivedAck 플래그를 확인하며 상태 전이를 수행한다.

■ Tag 통신 절차 (이니시에이터)
태그는 앵커 A와 B를 순차적으로 폴링하며 통신을 주도하는 주체이다.

- ① POLL 송신 : 대상 앵커의 주소를 data[1]에 설정하고, DW1000Ng::startTransmit()을 호출하여 POLL 프레임을 전송한다.
- ② POLL_ACK 수신 및 기록 : 앵커로부터 POLL_ACK 수신 완료 후 timePollSent 와

timePollAckReceived 2개의 타임스탬프를 기록한다. 이 두 값은 다음 RANGE 프레임 전송 시 data[] 버퍼에 저장되어 앵커로 전달된다.

- ③ RANGE 전송 (지연 송신 포함) : 정확한 RANGE 송신 시각(timeRangeSent)을 계산하기 위해 라이브러리의 지연 송신 함수를 적용한다. 기록된timePollSent, timePollAckReceived, timeRangeSent 3개의 값을 패킹한 후 전송한다.
- ④ RANGE_REPORT 수신: 앵커에서 계산된 최종 거리 값(float)을 RANGE_REPORT 프레임의 데이터 버퍼로부터 복원하여 획득한다.

■ Tag 통신 절차 (이니시에이터)
앵커는 태그가 보낸 메시지를 파싱하여 자신의 주소(data[1])와 일치할 경우에만 응답하는 선택적 응답 구조를 갖는다.

- ① POLL 수신 및 기록: 태그로부터 POLL 수신 완료 시 timePollReceived를 기록하고 즉시, POLL_ACK를 회신한다.
- ② POLL_ACK 송신 기록: POLL_ACK 송신 직후 timePollAckSent를 기록한다.
- ③ RANGE 수신 및 타임스탬프 복원: RANGE 수신 완료 시 timeRangeReceived를 기록하고, 동시에 태그로부터 수신한 3개의 타임스탬프(timePollSent, timePollAckReceived, timeRangeSent)를 복원하여 총 6개의 타임스탬프를 확보한다.
- ④ 거리 계산 및 RANGE_REPORT 송신: 확보된 6개 타임스탬프를 DW1000NgRanging::computeRangeAsymmetric()함수에 입력하여 TOF 기반의 거리를 계산한다. 이 거리 값은 칩 오차 보정 함수를 거친 후, RANGE_REPORT에 담겨 태그로 전송됨과 동시에 UART를 통해 라즈베리 파이로 출력된다.

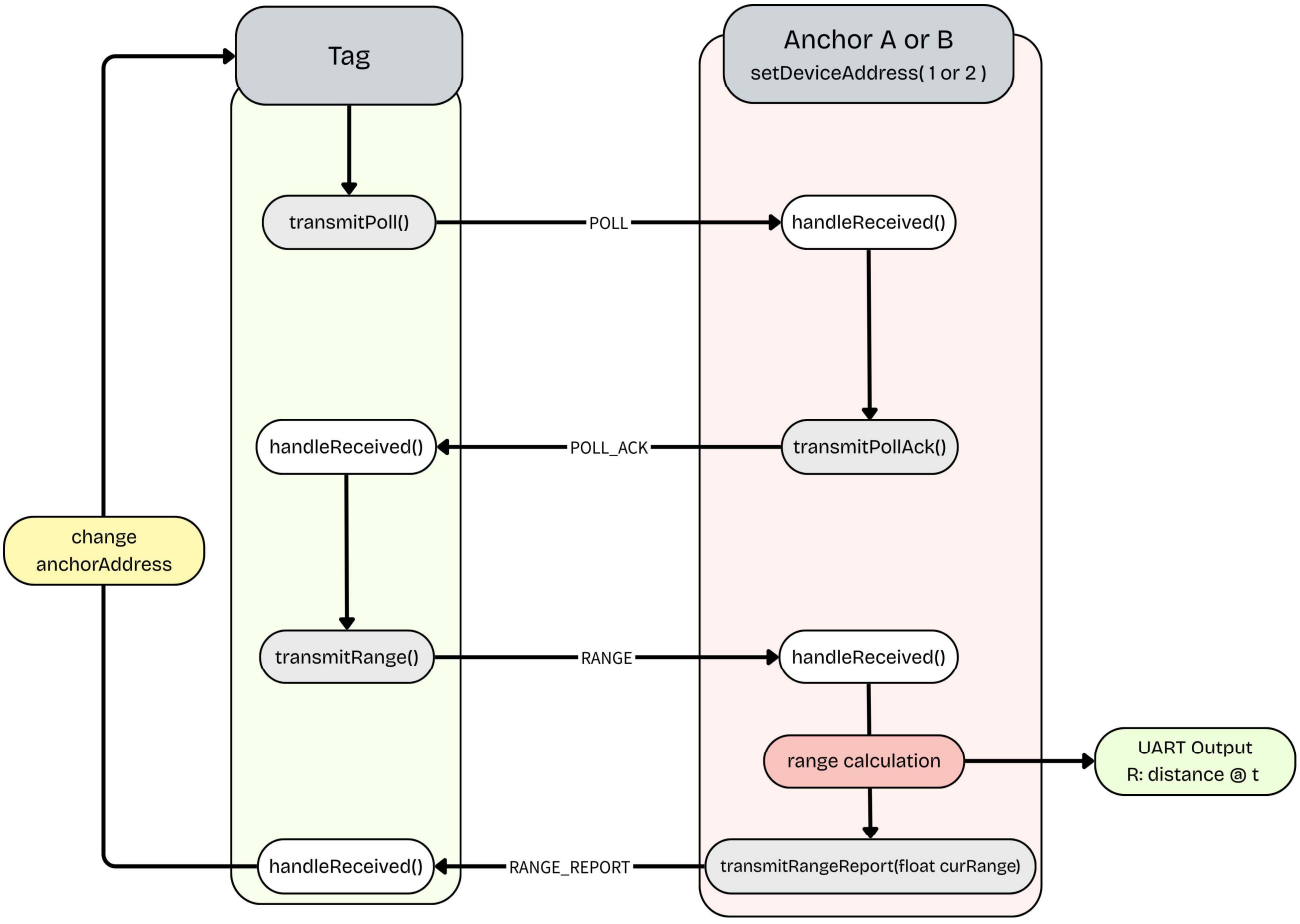
<ADS-TWR 프로토콜 과정 4>. 1대 2 폴링 구조 구현

기존 라이브러리의 1대 1 TWR 예제는 주소 필드가 없어 다자간 통신이 불가능했다. 본 연구에서는 이를 1대 2 구조로 확장하기 위해 다음과 같이 수정하였다.

- 데이터 버퍼 확장 및 주소 필드 추가: data[1]에 대상 앵커의 주소를 지정하는 필드를 추가하였다.

- 선택적 폴링: 태그는 통신이 완료될 때마다
  currentAnchorIndex = (currentAnchorIndex + 1) % 2; 로 타겟 주소를 변경하여
  Anchor A -> Anchor B -> Anchor A... 순으로 순차 폴링한다.

- 거리 갱신 주기: 이러한 순차 폴링을 통해 태그는 약 20Hz 속도로 두 앵커의 거리값을 갱신한다. 전체 ADS-TWR 1대 2 폴링 구조는 <그림 2.4.3> 에 나타나 있다.



<그림 2.4.3> Tag가 Anchor A/B를 번갈아 폴링하는 전체 프로토콜

<ADS-TWR 프로토콜 과정 5>. Raspberry Pi로의 결과 전송

앵커 는 ADS-TWR 프로토콜을 통해 최종 거리를 계산한 후, 이 결과를 라즈베리 파이로 전송한다. 계산된 거리값은 UART 포트를 통해 라즈베리 파이로 송신된다. 전송 데이터는 라즈베리 파이의 안정적인 파싱을 위해 다음과 같은 고정된 문자열 형식을 사용한다.

Format : " R: <Distance> @ <Timestamp> "

- 정상 데이터: R: 1.243 @ 514230
- 오류 및 신호 없음: R: 9999 @ t (오류) 또는 R: nosig @ t (신호 없음)

각 ESP32 앵커는 USB-UART 변환 칩을 통해 라즈베리 파이의 독립된 시리얼 포트(/dev/esp_A, /dev/esp_B)로 데이터를 송신하며, 라즈베리 파이는 이 두 포트로부터 거리 값을 병렬적으로 수신하여 상위 제어 로직에 사용한다.

(2) UWB 거리 데이터 후처리

ESP32 앵커로부터 UART를 통해 수신된 원시 거리 데이터(d_raw)는 실내 환경의 다중 경로 페이딩이나 통신 지연으로 인해 노이즈와 오차를 포함할 수 있다. 따라서 라즈베리 파이에서는 신뢰성 있는 거리 추정을 위해 <그림 2.4.4>와 같이 ① 유효성 검사 -> ② 스파이크 검출 -> ③ 이동평균 필터링 -> ④ 오차 보정의 4단계 후처리 알고리즘을 적용하였다.

1) 유효성 검사 (Valid Check)

수신된 문자열을 파싱한 직후, 데이터의 물리적 타당성을 일차적으로 검증한다. 다음 조건에 해당하는 데이터는 무효로 처리하여 폐기하고, 알고리즘의 입력으로 사용하지 않는다.

- 통신 오류: "9999" (ADS-TWR 실패 코드) 또는 "nosig" (신호 없음)
- 범위 오류: 음수 값 또는 물리적 임계치(본 연구에서는 5m)를 초과하는 비정상적인 값

2) 스파이크 검출 및 버퍼 관리 (Spike Detection & Buffer Reset)

유효성이 검증된 데이터(d_n)는 직전 수용된 데이터(d_n-1)와의 차이를 비교하여 스파이크 여부를 판단한다. 임계 값은 0.3m로 설정하였으며, 동작 로직은 다음과 같다.

- 정상 범위 ($|d_n - d_{n-1}| < 0.3$): 연속적인 데이터로 판단하여 필터 버퍼(history)에 추가한다.
- 급격한 변화 ($|d_n - d_{n-1}| \geq 0.3$): 사용자의 급격한 위치 이동으로 간주한다. 이 경우, 과거 데이터의 관성이 현재 추정을 방해하지 않도록 버퍼를 즉시 초기화(Clear)하고 현재 값부터 다시 수집을 시작한다.

이러한 조건부 버퍼 초기화 기법은 단순한 노이즈 제거를 넘어, 사용자의 급격한 기동 시 이동평균 필터가 가지는 고유한 지연 현상을 최소화하여 시스템의 반응 속도를 높이는 데 기여한다.

3) 이동평균 필터 (Moving Average Filter)

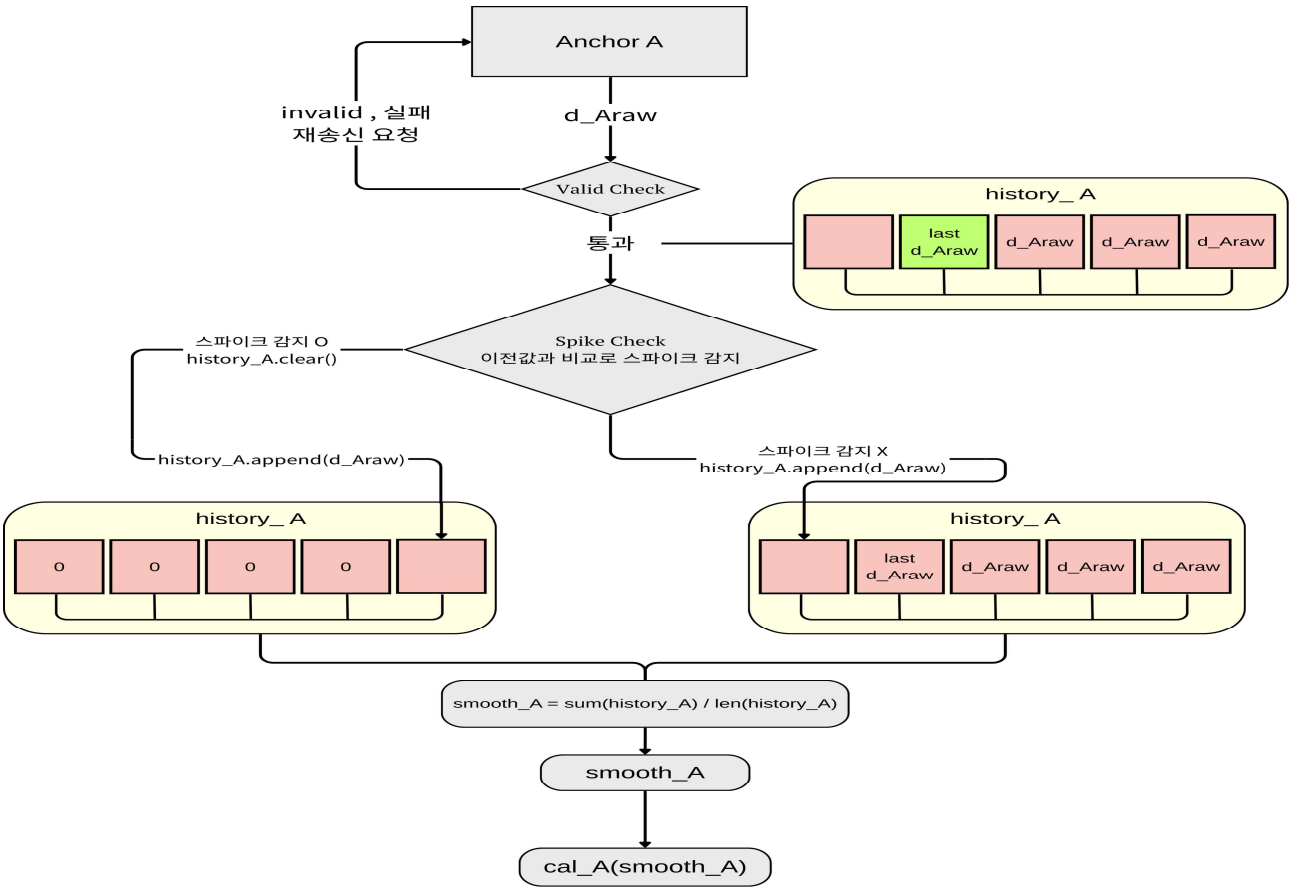
스파이크 검출 과정을 통과한 최근 N개의 데이터에 대해 <식 2.4.1>과 같은 이동평균 필터을 적용하여 고주파 노이즈를 제거하고 거리 값을 평활화(Smoothing)한다. 본 시스템에서는 응답성과 안정성의 균형을 위해 윈도우 크기 N=5를 적용하였다.

$$d_{smooth} = \frac{1}{N}\sum_{i=1}^{N} d_i \qquad \text{<식 2.4.1>}$$

4) 오차 보정 (Calibration)

최종적으로 UWB 모듈의 하드웨어 특성(안테나 지연, 회로 지연 등)을 보상하기 위한 보정 과정을 거친다. 앵커 A와 앵커 B는 설치 위치 및 하드웨어 편차에 따라 서로 다른 오차 특성을 보이므로, 실측 데이터 기반의 3차 다항식 피팅(Polynomial Fitting)모델을 각 앵커에 개별적으로 적용하여 최종 거리 d_A, d_B를 산출한다. 이 과정을 거쳐 정제된 두 거리 값은 이어지는 삼각측량 알고리즘의 입력으로 사용된다.

본 절에서 제안한 유효성 검사 및 이동평균 필터링 알고리즘의 실제 노이즈 제거 성능과 스파이크 검출 효과의 분석 결과는 제3장 실험 및 결과에서 기술한다.



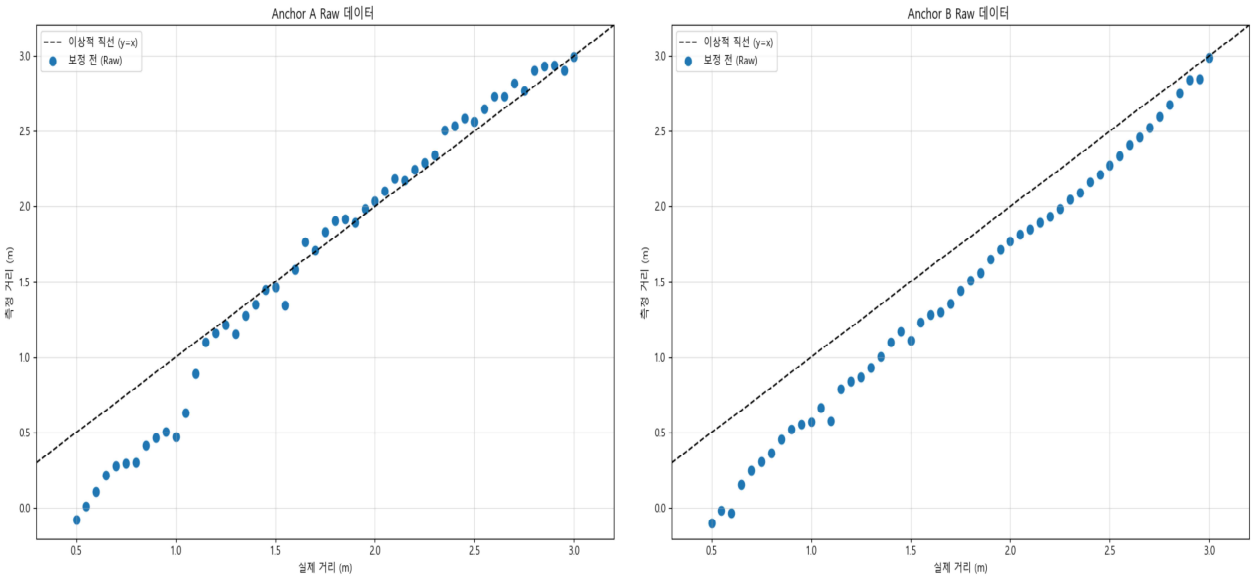
<그림 2.4.4> 데이터 후처리 과정

(3) 보정 알고리즘

스마트 카트 시스템의 UWB 거리 측정은 ADS-TWR 기반으로 이루어지지만, 실제 환경에서는 DWM1000 하드웨어 간 안테나 지연, 편차, 케이블 및 PCB 레이아웃 특성, 다중경로(Multipath) 반사 등의 요인으로 인해 측정값이 실제 거리와 선형적으로 일치하지 않는 비선형 왜곡이 발생한다. 따라서 앵커에서 획득한 후처리한 거리값(smooth_A, smooth_B)을 그대로 사용할 수 없으며, 라즈베리 파이 측에서 이를 실제 거리로 맵핑하는 보정(Calibration) 과정이 필수적이다. 본 절에서는 데이터 수집, 모델링, 그리고 앵커별 특성을 반영한 적응형 보정 전략을 기술한다.

1) 보정 데이터 수집 과정

정확한 보정식 도출을 위해 직선 경로와 고정된 높이를 유지한 실내 환경에서 실험을 수행하였다. 측정 거리는 0.50 m부터 3.00 m까지 0.05 m 간격으로 설정하여 총 51개 지점에서 데이터를 수집하였다. 각 지점마다 100회의 반복 측정을 수행하였으며, 지점별 측정값의 평균 편차는 약 0.03 m 수준으로 나타났다. 이에 따라 각 지점의 거리 값은 반복 측정 결과의 평균값을 대표값으로 사용하였다. <그림 2.4.5>는 고정된 각 포인트에서 구한 측정값과 실제 거리 값 의 차이를 나타낸다.



<그림 2.4.5> 앵커 A, 앵커 B 수집한 거리 데이터

앵커 A, B 모두 0.5 ~ 1.5 m 근거리 구간에서는 안테나 지연 및 반사파의 영향으로 큰 비선형 오차가 관찰되지만, 1.5 m 이후 구간에서는 비교적 선형적인 특성을 보였다. 이러한 비선형성을 효과적으로 보정하기 위해 단순 선형 회귀 대신 3차 다항식 회귀 모델을 적용하였다.

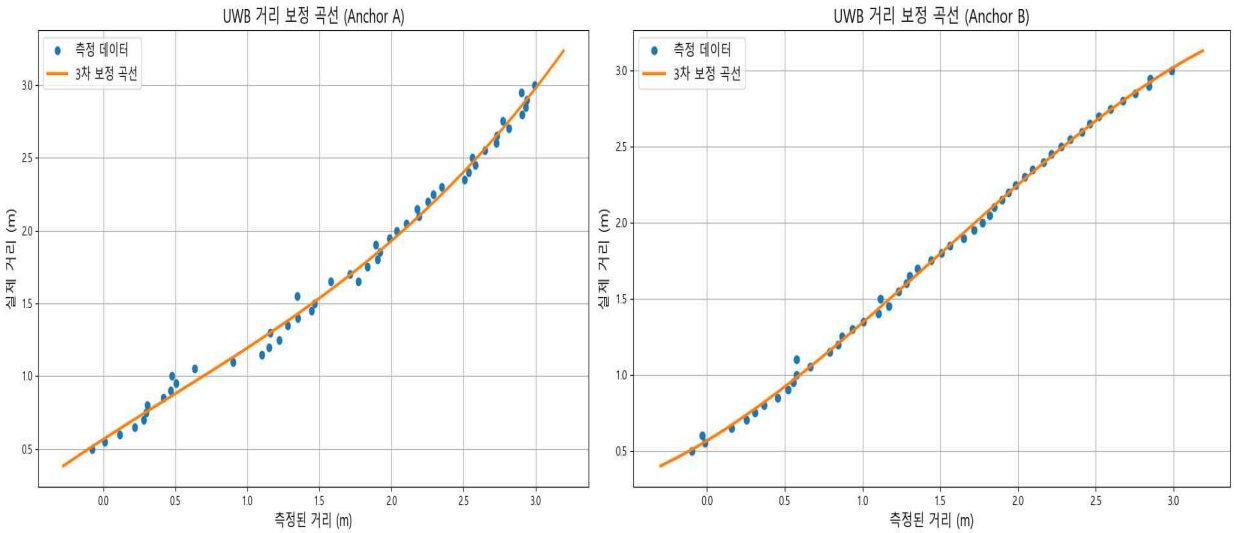
Python의 numpy.polyfit()을 이용하여 도출된 각 앵커의 보정식은 다음과 같다.

$$d_{calA} = 0.034143x^3 - 0.048561x^2 + 0.642064x + 0.566791$$ <식 2.4.2>

$$d_{calB} = -0.044688x^3 + 0.200146x^2 + 0.620484x + 0.565925$$ <식 2.4.3>

여기서 x는 측정된 원시 거리에 후처리를 적용한 d_{smooth} , d_{cal} 은 보정된 거리이다.

<그림 2.4.6>는 보정식으로부터 나온 보정곡선과 측정 데이터의 관계를 그래프로 보여준다.



<그림 2.4.6> 앵커 A, 앵커 B 거리보정 곡선

2)오차 분석

<표 2.4.3> 앵커 A 오차율 비교

거리 구간	보정 전 평균 오차율	보정 후 평균 오차율	보정 전 최대 오차율	보정 후 최대 오차율
0.5-1.5m	41.5%	5.28%	115.6%	13.6%
1.55-3.0m	3.4%	1.92%	13.3%	7.98%

<표 2.4.4> 앵커 B 오차율 비교

거리 구간	보정 전 평균 오차율	보정 후 평균 오차율	보정 전 최대 오차율	보정 후 최대 오차율
0.5-1.5m	51.2%	2.9%	105.3%	10.87%
1.55-3.0m	11.3%	0.9%	21.9%	2.27%

도출된 보정 식을 적용하여 오차율을 분석한 결과는 <표 2.4.3>, <표 2.4.4>과 같다. 앵커 A

와 앵커 B 모두 전 구간(0.5 ~ 3.0 m)에서 보정식 적용 시 오차가 현저히 감소하는 것을 확인할 수 있었고, 단일 모델 적용이 유효함을 확인하였다.

■ **공통 안전장치 (Clamping)**

보정된 값이 물리적으로 불가능한 수치로 튀는 것을 방지하기 위해 최소 거리(0.5 m) 미만 값은 0.5m로 고정(Clamp)하며, 3.0 m를 초과하는 원시 데이터는 보정식의 발산 위험이 있으므로 보정 없이 사용한다.

3) 최종 보정 알고리즘

위 전략을 수식으로 표현하면 다음과 같으며, 이를 Python 함수 apply_calibration()으로 구현하여 실시간 거리를 계산한다.

$$d_A = \begin{cases} 0.5, & \text{if} \quad d_{calA} < 0.5 \\ d_{calA}, & \text{if} \quad 0.5 \le d_{calA} \le 3.0 \\ d_{raw}, & \text{if} \quad d_{raw} \ge 3.0 \end{cases} \qquad \text{<식 2.4.4>}$$

$$d_B = \begin{cases} 0.5, & \text{if} \quad d_{calB} < 0.5 \\ d_{calB}, & \text{if} \quad 0.5 \le d_{calB} \le 3.0 \\ d_{raw}, & \text{if} \quad d_{raw} \ge 3.0 \end{cases} \qquad \text{<식 2.4.5>}$$

이처럼 앵커별 특성에 맞게 구한 보정식을 적용한 최적화된 거리 값 d_A, d_B는 다음 단계인 삼각측량 알고리즘의 입력으로 전달되어 사용자의 위치를 정밀하게 추정하는 데 사용된다.

(4) 두 앵커 기반 삼각측량 알고리즘

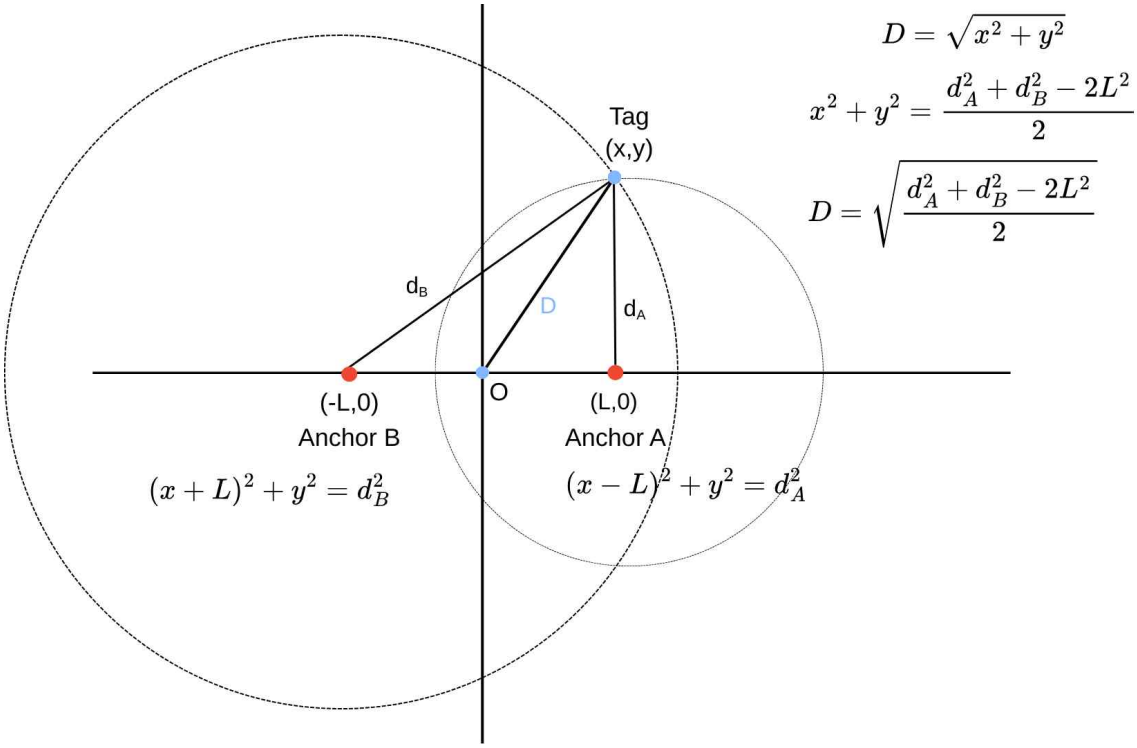
본 연구의 스마트 카트 시스템은 카트에 고정된 앵커 A, B와 태그 사이의 거리를 이용하여, 카트 중심으로부터 사용자까지의 실제 직선거리를 계산한다. 일반적인 실내 위치 측위 시스템이 복잡한 연산을 사용하는 것과 달리, 본 시스템은 기하학적 제약 조건인 앵커 A, B 간 고정된 거리를 활용하여 연산 부하가 매우 적은 직접 거리 계산식을 도출하여 적용하였다. 이 방식은 다음과 같은 장점이 있다.

- 실시간성: 연산 과정이 단순하여 라즈베리 파이 환경에서 50Hz 이상의 고속 처리가 가능하다.

- 강인성: 두 앵커 사이의 거리가 좁은 환경에서도, 좌우 치우침 에 의한 오차 영향이 비교적 적다.

1) 좌표계 설정 및 기하학적 모델링

시스템 모델링을 위해 카트의 중심점을 원점 (0,0)으로 설정하였다. 두 앵커 A와 B는 카트 전면부에 0.42m 간격으로 고정되어 있으므로, 앵커의 좌표는 원점으로부터 좌우 대칭인 위치에 정의된다. 카트 중심 좌표계 와 앵커 배치 모델은 <그림 2.4.7>과 같다.



<그림 2.4.7> 카트 중심 좌표계 및 앵커 배치 모델

- 카트 중심 : (0,0)
- 앵커 A 좌표: (L, 0)
- 앵커 B 좌표: (-L, 0)
- 태그 좌표: (a, b)
- 앵커 간격 (2L): 0.42 m (즉, L = 0.21 m)

2) 거리 계산 수식 유도

태그의 위치 (a, b)는 두 앵커를 중심으로 하고 반지름이 각각 d_A, d_B인 두 원의 교점으로 표현된다. 이를 원의 방정식으로 나타내면 다음과 같다.

$$(x-L)^2 + y^2 = d_A^2 \quad (1)$$

$$(x+L)^2 + y^2 = d_B^2 \quad (2)$$

본 연구의 목적은 태그의 좌표 (a b) 자체를 구하는 것보다, 카트 중심에서 태그까지의 직선거리 D를 구하는 것이다. 원점과 태그 사이의 거리는 $D = \sqrt{x^2 + y^2}$ 이므로, 위 두 식을 전개하여 $x^2 + y^2$ 항을 직접 도출할 수 있다. (1) 과 (2) 를 전개하면:

$$x^2 - 2xL + L^2 + y^2 = d_A^2$$
$$x^2 + 2xL + L^2 + y^2 = d_B^2$$

두 식의 양변을 더하면 x에 대한 1차항 $(-2xL,\ +2xL)$이 소거된다.

$$(x^2 + y^2 + L^2 - 2xL) + (x^2 + y^2 + L^2 + 2xL) = d_A^2 + d_B^2$$
$$2(x^2 + y^2) + 2L^2 = d_A^2 + d_B^2$$

이를 $x^2 + y^2$ 에 대해 정리하면 다음과 같은 최종거리 수식을 얻는다.

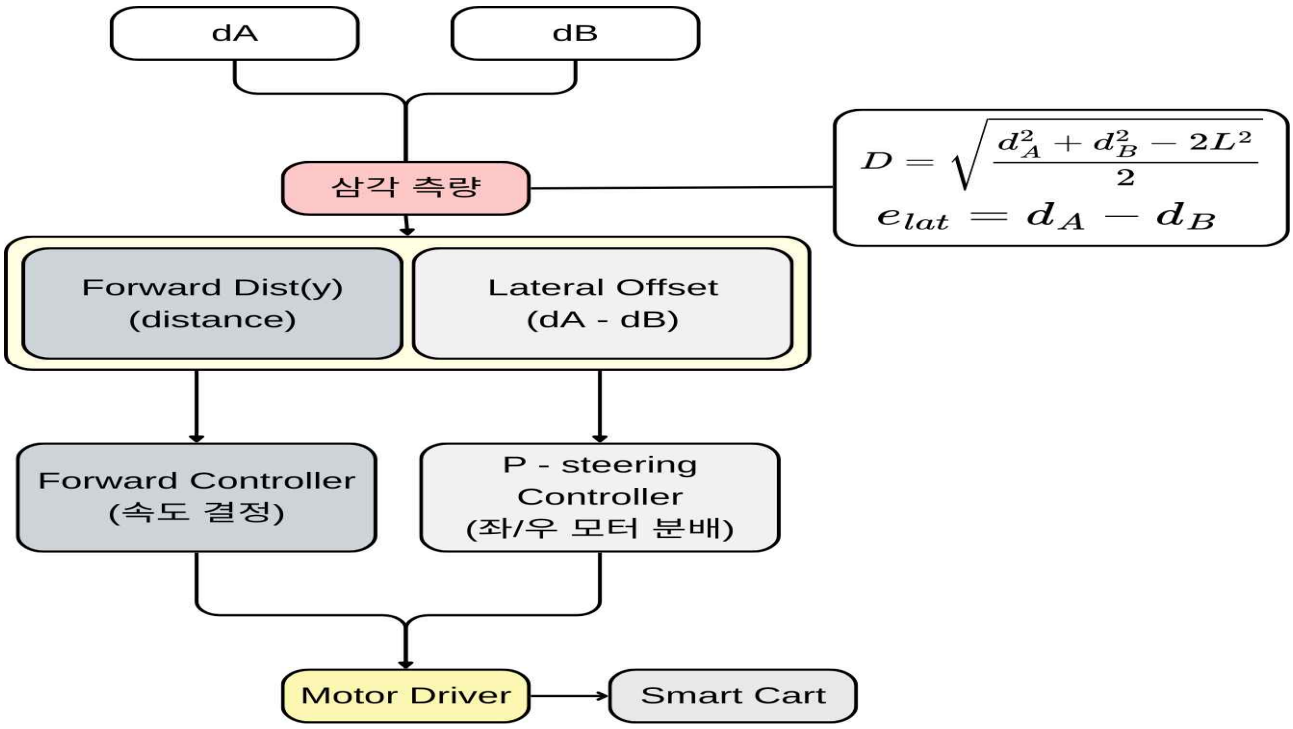
$$x^2 + y^2 = \frac{d_A^2 + d_B^2 - 2L^2}{2}$$

따라서, 카트 중심과 태그 사이의 최종거리 D_{center} 는 <식 2.4.6>과 같이 계산된다.

$$D_{center} = \sqrt{\frac{d_A^2 + d_B^2 - 2L^2}{2}} \qquad \text{<식 2.4.6>}$$

(5) 카트 추종 제어 알고리즘

본 연구의 스마트 카트는 UWB 시스템을 통해 획득한 태그의 위치 정보를 기반으로, 카트의 전진 속도와 조향각을 실시간으로 제어한다. 제어기의 입력으로는 앞선 2.4.6절에서 도출한 카트 중심 거리(D)와 두 앵커 간의 거리 편차(e)를 사용하며, 전체 제어 구조는 <그림 2.4.8>과 같이 속도 제어기와 조향 제어기가 병렬로 동작하는 구조를 갖는다.



<그림 2.4.8> 추종 제어 구조

1) 제어 입력 정의

제어 알고리즘은 다음 두 가지 상태 변수를 기반으로 동작한다.

- 정면 거리 (Forward Distance) :

$$D = \sqrt{\frac{d_A^2 + d_B^2 - 2L^2}{2}}$$

- 좌우 편차 (Lateral Offset) :

$$e_{lat} = d_A - d_B$$

여기서 e_lat 가 0에 수렴하도록 제어하면 카트는 사용자를 정면에 위치시키게 되며, D를 목표 거리에 맞추면 일정한 간격을 유지하게 된다.

2) 전진 속도 제어

사용자와의 거리에 따라 카트의 기본 주행 속도(v_base)를 결정하기 위해, 연산 부하가 적고 반응성이 빠른 구간 기반 제어(Region-based Control) 기법을 적용하였다. 목표 추종 거리는 0.6m로 설정하였으며, 거리에 따른 속도 결정 로직은 다음과 같다.

$$v_{base} = \begin{cases} v_{HIGH}, & \text{if } D > 2.0\text{m} \\ \text{v}_{\text{MID}}, & \text{if } 0.6\text{m} < \text{D} \le 2.0\text{m} \\ 0 \ (\text{STOP}), & \text{if } D \le 0.6\text{m} \end{cases}$$

3) 조향 제어

카트가 사용자를 향해 방향을 전환하도록 두 앵커의 거리 차(e_lat)를 오차항으로 하는 P 제어(Proportional Control)를 수행한다. 조향을 위한 제어 입력 u(t)는 다음과 같이 정의된다.

$$u(t) = K_p \times e_{lat}(t) \qquad \text{<식 2.4.7>}$$

여기서 K_p는 비례 이득(Proportional Gain)으로, 본 연구에서는 70으로 설정하였다. 직진 주행 시 불필요한 조향 떨림을 방지하기 위해, 오차의 절댓값이 특정 임곗값 (Dead Band, 0.15m) 미만일 경우 조향 입력을 0으로 처리한다.

$$u_{final} = \begin{cases} 0, & \text{if } |e_{lat}| < 0.15 \\ u(t), & otherwise \end{cases}$$

최종적으로 좌우 바퀴의 모터 속도(v_L, v_R)는 차동 구동(Differential Drive) 방식에 따라 기본 속도(v_{base)에 조향 입력(u_final)을 가감하여 결정된다.

$$v_L = v_{base} - u_{final}$$
$$v_R = v_{base} + u_{fianal}$$

4) 모터 구동 및 안전 제한

계산된 좌우 바퀴의 속도는 L298N 모터 드라이버를 통해 PWM 듀티비(Duty Ratio) 로 변환되어 모터에 인가된다. 이때, 모터의 물리적 한계와 저속 구간에서의 구동 불능 현상을 방지하기 위해 다음과 같은 포화 및 최소 속도 제한 로직을 적용하였다.

- 최대 속도 제한 : $|v| > 100 \Longrightarrow v = 100$ (PWM MAX)
- 최소 속도 제한 : $|v| < v_{\min} \Longrightarrow v = v_{\min}$ (구동 시작 전압 확보)

이러한 제어 알고리즘은 복잡한 연산 없이도 50 Hz 이상의 주기로 고속 수행되어, 사용자의 움직임에 지연 없이 반응하는 부드러운 추종 주행을 가능하게 한다.

(6) 장애물 감지 및 안전 제어

스마트 카트는 사용자 추종 중에 발생할 수 있는 충돌 사고를 방지하기 위해, 전방 장애물을 능동적으로 탐지하고 정지 - 회피 - 복귀의 3단계 안전 제어 프로세스를 수행한다. 이를 위해 카트 전면부의 좌(Left), 중앙(Center), 우(Right) 방향에 배치된 3개의 초음파 센서 를 활용하며, 감지된 거리 데이터를 기반으로 최적의 회피 경로를 결정한다.

1) 거리 측정 및 노이즈 필터링

초음파 센서는 트리거 펄스 인가 후 에코 신호가 반환되는 시간을 측정하여 거리를 산출한다. 하지만 초음파 센서는 반사 각도나 환경적 요인으로 인해 간헐적인 노이즈나 튀는 값이 발생할 수 있다. 오탐지로 인한 불필요한 정지를 방지하기 위해, 본 시스템은 연속 감지 카운터 기법을 적용하였다.

- 판단 로직: 단일 측정값이 임계값 이하라고 해서 즉시 장애물로 판단하지 않고, 5회 연속으로 임계값 이하가 감지될 때만 유효한 장애물 이벤트(Obstacle Event)를 발생시킨다.

2) 장애물 회피 알고리즘

장애물 이벤트가 발생하면 시스템은 추종 제어를 일시 중단하고 장애물 회피 상태로 진입한다. 회피 알고리즘은 상황 판단 및 안전 확보를 위해 다음 6단계의 차례로 동작한다.

- 1. 즉시 정지 및 대기 : 관성에 의한 충돌을 막기 위해 모터를 즉시 정지시키고, 약 1.5초간 대기하며 센서 데이터가 안정화되기를 기다린다.

- 2. 재측정 및 유효성 검증 : 정지 상태에서 3개의 센서 값을 각각 3회씩 재측정하여 평균을 구한다. 이때 모든 센서값이 임계값을 벗어났다면(장애물 소멸), 즉시 정상 추종 모

드로 복귀한다.

- 3. 후진 기동 : 장애물이 여전히 존재할 경우, 회전 반경 확보를 위해 일정 거리만큼 후진한다.

- 4. 방향 결정 : 후진 후 좌/우 센서의 평균 거리 값을 비교하여 여유 공간이 더 넓은 쪽으로 회전 방향을 결정한다.
  - $d_{left} < d_{right}$ -> 우회전
  - $d_{left} \geq d_{right}$ -> 좌회전

- 5. 회전 및 탐색 : 선택된 방향으로 제자리 회전을 수행하며, 전방 공간이 확보될 때까지 탐색한다.

- 6. 복귀 주행 : 전방이 개방되면 일정 시간 직진하여 장애물 구간을 벗어난 후, 다시 UWB 추종 제어 루프로 복귀한다.

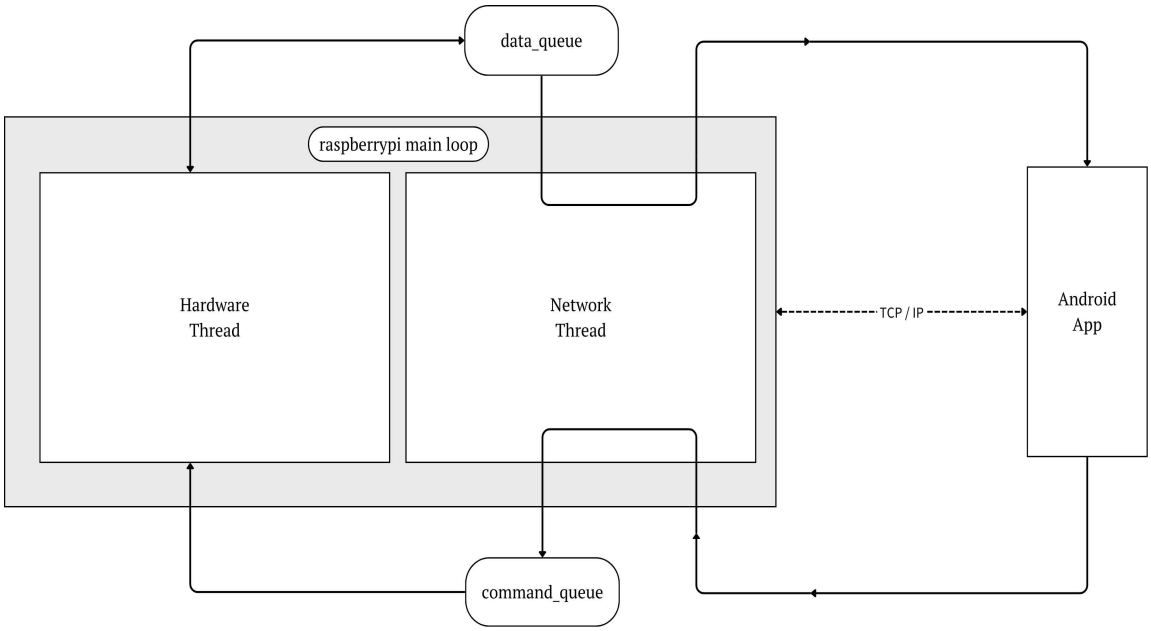
3) 안전 대기모드

만약 위 회피 과정을 수행했음에도 불구하고 여전히 장애물이 감지되거나 회피할 수 없는 좁은 공간에 갇힌 경우, 시스템은 무리한 주행을 시도하지 않고 안전 대기 상태(WAITING_FOR_RESUME)로 전환된다. 이 상태에서는 카트가 완전히 정지하며, 사용자가 상황을 확인하고 스마트폰 앱에서 [RESUME] 버튼을 눌러 수동으로 해제하기 전까지 대기함으로써 2차 사고를 방지하도록 설계되었다.

(7) 네트워크 기반 명령 및 데이터 흐름

앞서 기술된 UWB 센서 처리 및 하드웨어 제어 로직은 라즈베리 파이 내부에서 독립적으로 수행되지만, 사용자(스마트폰 앱)와의 상호작용을 위해서는 외부와의 통신 인터페이스가 필수적이다. 본 절에서는 스마트 카트 시스템의 네트워크 계층(Network Layer) 구조와 데이터 흐름을 기술한다.

본 시스템은 Wi-Fi 기반 TCP/IP 통신을 통해 양방향 데이터를 실시간으로 송수신하며, 라즈베리 파이 내부에서는 하드웨어 제어 스레드(Hardware Thread)와 네트워크 스레드(Network Thread)가 분리되어 동작하는 멀티 스레드(Multi-thread) 구조를 채택하였다. 두 스레드는 직접적인 메모리 공유 대신 스레드 안전 큐(Thread-safe Queue)를 통해 비동기적으로 메시지를 교환함으로써 시스템의 안정성과 반응성을 확보하였다. <그림 2.4.9>는 네트워크와 하드웨어 제어 두 스레드를 사용하는 구조와 스레드 간의 데이터를 주고받는 흐름을 보여준다.



<그림 2.4.9> 내부 스레드 구조 및 데이터 흐름도

1) 전체 통신 구조 및 컴포넌트

스마트 카트의 통신 아키텍처는 크게 4개의 컴포넌트로 구성되며, 각 요소는 명확히 분리된 역할을 수행한다.

- 1. 스마트폰 앱 (TCP Client): 사용자 인터페이스 역할을 하며, 제어 명령(START, STOP 등)을 전송하고 수신된 카트의 상태(거리, 모드)를 시각화한다.
- 2. 네트워크 스레드 (TCP Server): 앱과의 연결을 관리하고 소켓 통신을 전담한다. 수신된 명령은 command_queue에 적재하고, data_queue에 쌓인 상태 정보를 앱으로 전송
- 3. 하드웨어 스레드 (System Controller): 실제 카트 제어(UWB 측정, 모터 구동, 장애물 감지)를 담당한다. 주기적으로 command_queue를 확인하여 동작 모드를 변경하고, 수행 결과 및 상태 정보를 data_queue에 기록한다.
- 4. 메시지 큐 (IPC Mechanism): 두 스레드 간의 데이터 교환 통로로, 생산자-소비자 패턴을 통해 데이터 동기화 문제를 해결한다.

2) 네트워크 스레드 동작 로직

네트워크 스레드는 라즈베리 파이를 TCP 서버(Host)로 구동하며, 클라이언트(앱)의 연결 요청을 대기 한다. 연결 수립 후에는 handle_client() 함수가 실행되어 다음과 같은 비동기 송수신 루프를 수행한다.

- 수신 (Rx): 소켓 버퍼를 주기적으로 확인하여 클라이언트로부터 전송된 데이터가 있으면 이를 디코딩하여 command_queue에 삽입(Push)한다.
- 송신 (Tx): data_queue를 확인하여 하드웨어 스레드가 생성한 최신 상태 정보가 있다면 이를 인출(Pop) 하여 클라이언트로 전송한다.

- 예외 처리: 네트워크 끊김이나 비정상 종료 발생 시, 이를 감지하여 소켓을 안전하게 닫고 STOP 명령을 큐에 강제로 삽입함으로써 카트가 폭주하지 않고 안전하게 정지하도록 설계하였다.

3) 큐 기반 메시지 프로토콜

두 스레드는 정의된 텍스트 기반 프로토콜을 사용하여 정보를 교환한다.

■ 제어 명령 흐름 (App -> Raspberrypi(Hardware Thread))

스마트폰 앱에서 버튼 입력 시 command_queue로 전송되는 명령은 다음과 같으며, 하드웨어 스레드는 command_queue 읽어 내부 플래그 변수인 (tracking, waiting)를 갱신한다.

- START : 추종 제어 시작 (tracking = True)
- STOP : 제어 중단 및 정지 (tracking = False)
- RESUME : 안전 대기 상태 해제 및 복귀

■ 상태 정보 흐름 (Raspberrypi(Hardware Thread) -> App)

하드웨어 스레드는 제어 주기마다 현재 거리와 동작 모드를 포맷팅하여 state_queue로 전송한다. 네트워크 스레드는 state_queue를 앱에게 보낸다.

포맷 : DIST:<Distance> MODE:<State>

- 예시:
  - DIST:0.42 MODE:TRACKING (정상 추종 중)
  - DIST:1.10 MODE:PROXIMITY_STOP (근접 정지)
  - DIST:--- MODE:TRACKING_FAIL (통신 오류)

이러한 구조를 통해 네트워크 지연이나 연결 끊김이 발생하더라도 하드웨어 제어 루프는 영향을 받지 않고 독립적으로 안전하게 동작할 수 있다. 다음 절에서는 이 메시지 구조를 기반으로 구현된 상태 기반 제어(FSM) 시스템에 대해 상세히 기술한다.

(8) 상태 기반 제어

스마트 카트의 최종 주행 제어는 라즈베리 파이의 하드웨어 스레드에서 수행된다. 이 스레드는 UWB 및 초음파 센서 입력을 실시간으로 처리하고, 현재 상황에 적합한 모터 출력을 결정하는 이벤트 기반 유한 상태 기계(FSM)로 설계되었다. 앞선 2.4.7절에서 기술한 바와 같이, 제어 로직은 네트워크 스레드와 변수를 직접 공유하지 않고 명령 큐(command_queue)와 데이터 큐(data_queue)를 통해 비동기적으로 상호작용한다. 이는 외부 명령(App)과 내부 센서 이벤트가 결합 되어 시스템의 상태를 변화시키는 구조이다.

1) 상태 정의

시스템의 동작은 <표 2.4.5> 처럼 4가지 주요 상태와 장애물 회피를 위한 특수 동작 상태로 정의된다.

<표 2.4.5>

상태	동작	의미
WAITING	대기 상태	모터 출력을 차단하고 사용자 명령을 기다림. (초기 상태)
TRACKING	추종 상태	목표 거리를 유지하며 사용자를 따라 이동함.
PROXIMITY_STOP	근접 정지	사용자와의 거리가 안전거리(0.6m) 이내일 경우 일시 정지.
TRACKING_FAIL	추종 실패	센서 오류나 통신 두절로 거리 측정이 불가능한 경우 안전 정지.
(OBSTACLE_AVOID)	장애물 회피	장애물 감지 시 진입하는 임시 제어 시퀀스.

2) 입력 이벤트 처리 및 상태 전이

FSM은 외부 명령(Command) 과 내부 센서(Sensor) 두 가지 소스로부터 이벤트를 수신하여 상태를 전이시킨다.

■ 외부 명령 처리 (Command Queue)

네트워크 스레드로부터 전달된 명령은 FSM의 상위 동작 모드를 결정한다.

- START: WAITING -> TRACKING (추종 시작)
- STOP: 모든 상태 -> WAITING (즉시 정지)
- RESUME: 장애물 회피 후 안전 대기 상태 -> TRACKING (복귀)

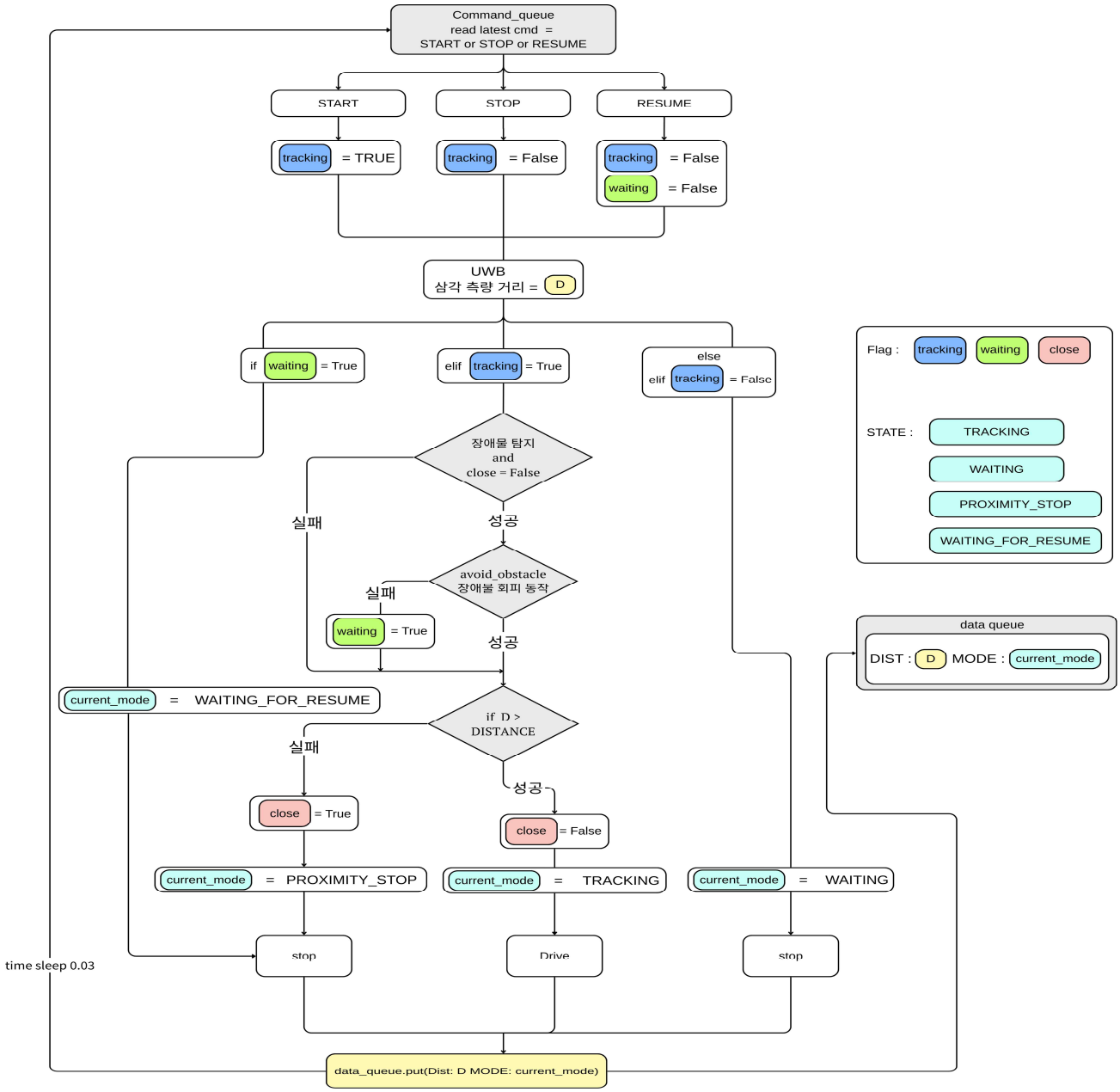
■ 내부 센서 데이터 처리

하드웨어 스레드는 매 제어 주기마다 센서 데이터를 갱신하며, 이는 상태 전이의 판단 근거가 된다.

- UWB 거리 데이터: 2.4.4절의 필터링과 2.4.6절의 삼각측량을 거쳐 계산된 최종 거리(D)를 사용
- 장애물 감지: 초음파 센서가 5회 이상 연속으로 장애물을 감지하면, OBSTACLE_AVOID 시퀀스를 트리거한다.

3) FSM 제어 로직 구조

하드웨어 스레드의 핵심 로직은 매 루프마다 [입력 갱신 -> 상태 판단 -> 제어 출력] 과정을 수행하며, 그 구조는 <그림 2.4.10> 과 같다.



<그림 2.4.10> FSM 플로우 차트

■ 안전 대기 점검 (Safety Check): 가장 먼저 waiting 플래그를 확인한다. 장애물 회피 후 대기 상태이거나 RESUME 명령을 기다리는 중이라면(waiting == True), 모든 제어를 중단하고 WAITING_FOR_RESUME 상태를 유지한다.

■ 추종 모드 진입 및 우선순위 판단 (Priority Decision): 사용자가 START 명령을 내려 tracking == True가 된 상태에서는 다음 순서대로 조건을 검사하여 최종 상태를 결정한다.

- 1 (장애물 체크) : 가장 먼저 초음파 센서값을 확인한다. 단, 카트가 이미 사용자와 너무 가까워 정지한 상태(close == True)가 아닐 때만 장애물 회피 로직이 동작한다. 장애물이 감지되면 avoid_obstacle() 시퀀스를 수행하며, 회피 실패 시 대기 상태로 전환된다.

- 2 (근접 정지) : 장애물이 없다면 사용자와의 거리(D)를 확인한다. 거리가 안전 임계값(본 연구에서는 0.6m) 미만일 경우, 충돌 방지를 위해 close 플래그를 True로 설정하고 PROXIMITY_STOP 상태로 전환하여 즉시 정지한다.

- 3 (정상 추종) : 위의 위험 요소가 모두 없을 때 비로소 TRACKING 상태가 된다. 이때 2.4.5절의 알고리즘에 따라 목표 거리 유지를 위한 속도 제어와 방향 조향이 수행된다.

■ 비활성 상태 (Idle): tracking과 waiting이 모두 False 인 경우(초기 상태 또는 STOP 명령 수신 시), 시스템은 WAITING 상태가 되어 대기한다.

■ 상태 보고 (Feedback): 매 루프의 마지막에는 결정된 최종 거리(D) 와 현재 상태(Current Mode) 를 DIST:1.25 MODE:TRACKING 형태의 메시지로 포맷팅하여 data_queue에 삽입한다. 이는 네트워크 스레드를 통해 스마트폰 앱으로 전송되어 사용자에게 실시간 피드백을 제공한다.

4) 상태 기반 제어의 장점

본 연구에서 채택한 이벤트 기반 FSM 구조는 다음과 같은 장점을 제공한다.

- 안정성 : 노이즈가 존재하는 센서 환경에서도 명확한 조건(Threshold, Counter)에 따라 상태가 구분되므로, 예측 불가능한 오동작을 방지할 수 있다.

- 디버깅 용이성: 현재 카트가 어떤 상태(MODE)에 있는지 앱을 통해 실시간으로 확인할 수 있어, 문제 발생 시 원인 파악이 쉽다.

- 확장성 : 모듈화된 상태 구조 덕분에 추후 '자율주행'이나 '경로 기록' 같은 새로운 기능을 추가할 때 기존 로직을 수정하지 않고 새로운 상태(State)만 추가하여 쉽게 기능을 확장할 수 있다.

(9) 안드로이드 앱 설계

본 절에서는 스마트 카트와의 무선 인터페이스를 담당하는 안드로이드 애플리케이션의 설계 구조를 기술한다. 앱은 Wi-Fi TCP/IP 통신을 기반으로 사용자의 제어 명령(START, STOP, RESUME)을 카트에 전달하고, 카트로부터 수신된 실시간 거리 및 주행 상태 정보를 시각화하여 제공한다. 앱 아키텍처는 안정적인 백그라운드 통신을 보장하기 위해 서비스 기반 구조(Service-based Architecture) 를 채택하였으며, MainActivity (UI), MyTcpService (포그라운드 서비스), TcpClient (통신 스레드)의 3계층으로 구성된다.

1) 핵심 컴포넌트 및 데이터 흐름

시스템의 데이터 흐름은 단방향이 아닌 양방향 반응형 구조 를 갖는다.

- 명령 흐름 (App -> Cart) : 사용자의 UI 조작 -> 서비스 -> TCP 클라이언트 -> 스마트 카트

- 상태 흐름 (Cart -> App): 스마트 카트 상태 메시지 -> TCP 클라이언트 -> LiveData 갱신 -> UI 자동 업데이트

■ 포그라운드 서비스 (MyTcpService)

안드로이드 운영체제의 메모리 관리 정책에 의해 백그라운드 통신이 강제로 종료되는 것을 방지하기 위해, TCP 연결 및 데이터 처리를 포그라운드 서비스(Foreground Service)로 구현하였다. 이 서비스는 화면이 꺼지거나 앱이 백그라운드로 전환되어도 연결을 유지하며, Binder 패턴을 통해 UI 액티비티와 통신한다.

■ 통신 스레드 (TcpClient)

메인 UI 스레드(Main Thread)의 차단(Blocking)을 방지하기 위해, 소켓 생성 및 입출력(I/O) 작업은 별도의 작업 스레드(Thread)에서 비동기적으로 수행된다.
수신된 메시지(DIST:xx MODE:yy)는 콜백 함수를 통해 서비스 계층으로 전달된다.

■ 반응형 UI (MainActivity & LiveData)

UI 계층은 서비스로부터 직접 데이터를 받는 대신, Observer 패턴 기반의 LiveData를 관찰(Observe)하여 화면을 갱신한다. 이를 통해 데이터 수신 시 별도의 UI 갱신 로직 없이도 텍스트 뷰와 버튼 상태가 자동으로 동기화된다.

2) 주요 기능 구현 로직

■ TCP 연결 및 상태 모니터링

앱 실행 시 서비스는 라즈베리 파이의 고정 IP 주소(ex: 10.195.104.70)와 포트(5000)로 소켓 연결을 시도한다. 연결 성공 여부는 connectionStatus LiveData를 통해 UI에 즉시 반영되어 '연결됨/해제됨' 상태를 표시한다.

■ 명령 전송 및 제어 (Command Interface)

사용자 입력은 상태 머신(FSM)의 이벤트 트리거 역할을 한다.

- START / STOP: 자동 추종 기능을 활성화하거나 정지한다.
- RESUME: 장애물 회피 등으로 인한 WAITING_FOR_RESUME 상태에서, 사용자가 안전을 확인한 후 정상 주행으로 복귀시키는 안전장치 명령이다.

■ 상태 파싱 및 UI 동기화

<표 2.4.6>

수신 모드 (MODE)	UI 동작 및 버튼 상태
TRACKING	거리 표시 갱신, [자동 추적 OFF] 버튼 활성화
WAITING	거리 표시 갱신, [자동 추적 ON] 버튼 활성화
WAITING_FOR_RESUME	[RESUME] 버튼 활성화, 경고 메시지 표시
PROXIMITY_STOP	거리 표시 갱신, 정지 상태 표시

카트로부터 수신된 메시지(DIST:1.19 MODE:TRACKING)는 파싱되어 UI 요소에 1:1로 매핑된다. <표 2.4.6> 는 수신모드에 따른 UI 동작 과 버튼의 상태를 보여준다.

이러한 구조는 2.4.8절의 하드웨어 상태 머신(FSM) 상태를 사용자에게 직관적으로 전달하며, 현재 카트 상황에 맞는 조작만을 허용하여 오조작을 방지한다. 본 앱 설계는 다음과 같은 기술적 이점을 제공한다.

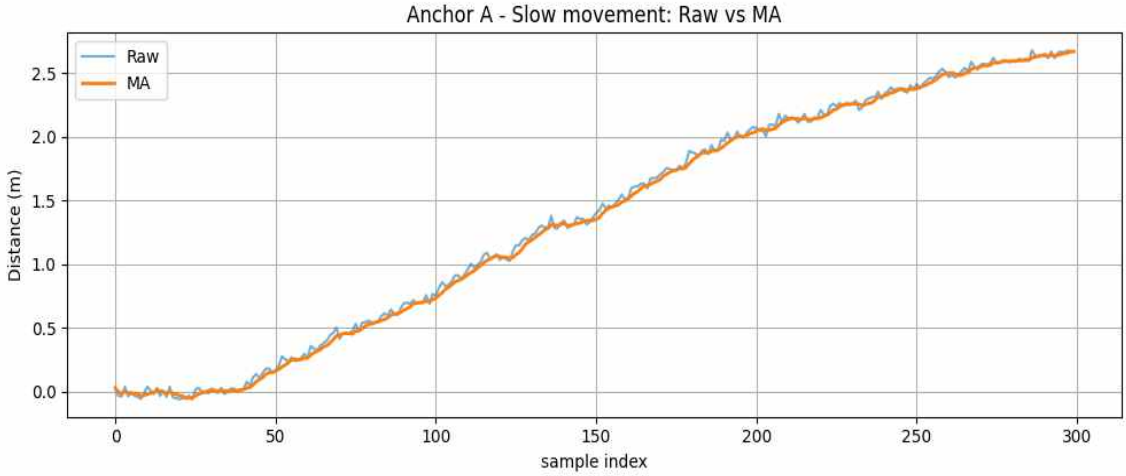
- 연속성 보장: 포그라운드 서비스 적용으로 화면 꺼짐이나 장시간 대기 시에도 연결 끊김 없는 안정적인 제어 환경을 제공한다.
- 실시간성: 별도의 폴링 과정 없이, 소켓 수신 이벤트가 LiveData를 통해 즉시 UI에 반영되는 이벤트 기반 아키텍처를 구현하였다.
- 디버깅 용이성: 텍스트 기반 프로토콜(String)을 사용하여 로그 확인 및 통신 상태 진단이 용이하며, 펌웨어 로직과 앱 로직 간의 정합성을 쉽게 검증할 수 있다.

2.5. 개발 결과 및 성능 평가

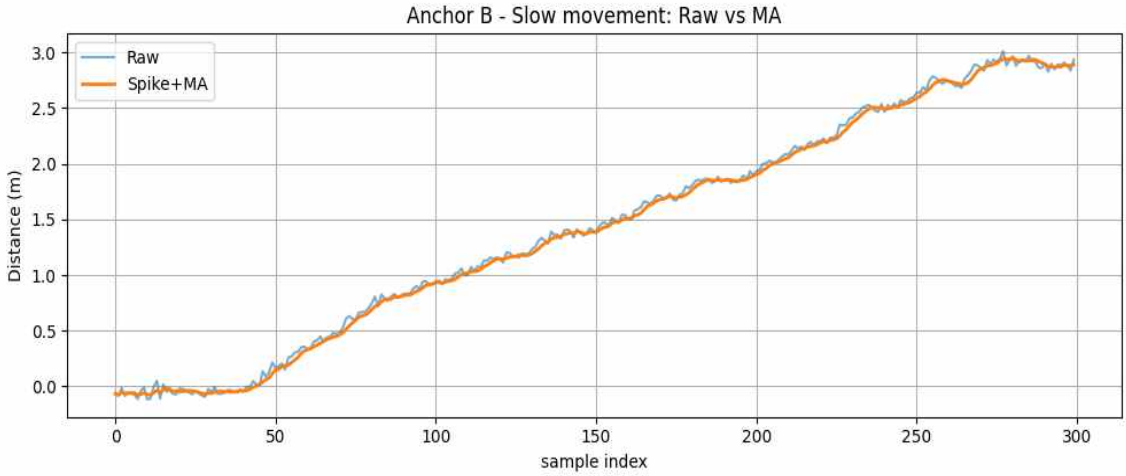
본 절에서는 2장에서 설계한 스마트 카트 시스템의 핵심 알고리즘(필터링, 거리 보정, 삼각측량)과 전체 통합 기능(추종 제어, 장애물 회피, 앱 연동)에 대한 실제 성능 검증 결과를 기술한다.

(1) UWB 거리 데이터 필터링 성능

1) 정적 및 저속 이동 시 노이즈 제거 (smoothing) :
태그를 약 2.5 m 거리에서 0.5 m까지 천천히 접근시키며 앵커 A/B로부터 약 300개의 원시 거리 데이터(Raw Data)를 수집하였다. <그림 2.5.1>과 <그림 2.5.2>는 각각 앵커 A와 앵커 B의 Raw 값과 이동평균 필터 출력(Smooth)을 비교한 그래프이다.

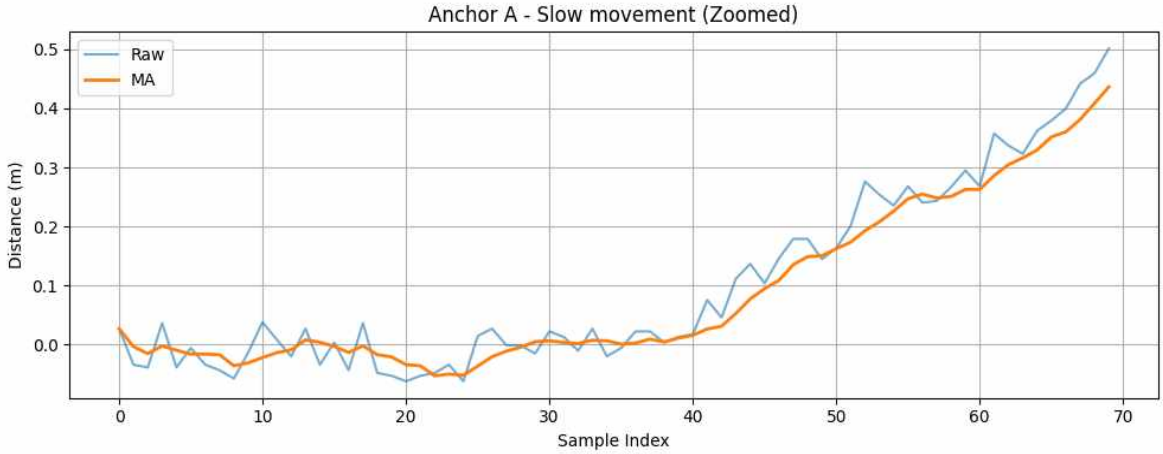


<그림 2.5.1> 앵커 A 정적 실험 : Raw 데이터와 이동평균 필터 결과 비교

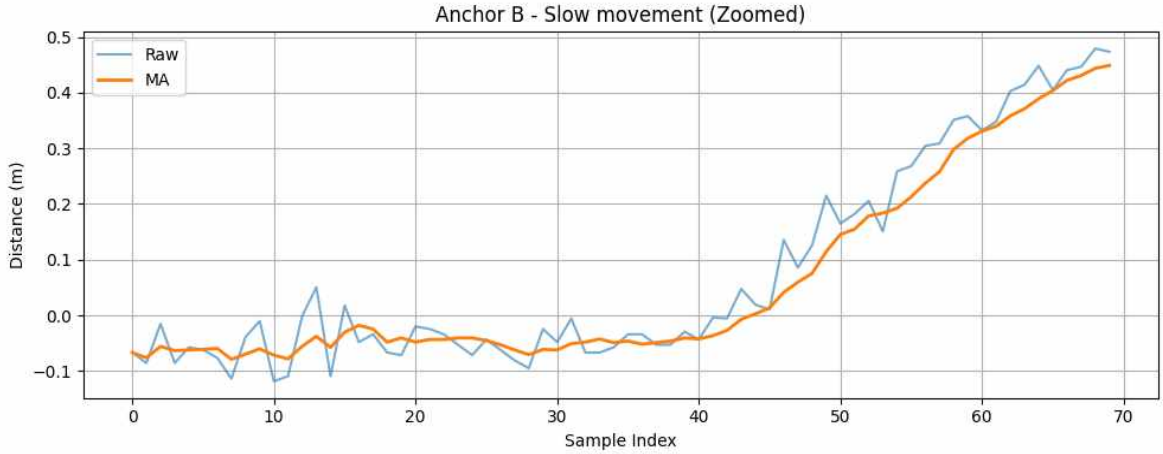


<그림 2.5.2> 앵커 B 정적 실험 : Raw 데이터와 이동평균 필터 결과 비교

샘플 인덱스 0~70 구간을 확대한 결과<그림 2.5.3> , <그림 2.5.4> 에서 볼 수 있듯이, 이동 평균 필터는 센서 고유의 고주파 잡음(Jitter)을 효과적으로 억제하여 부드러운 거리 추정을 가능하게 함을 확인하였다.



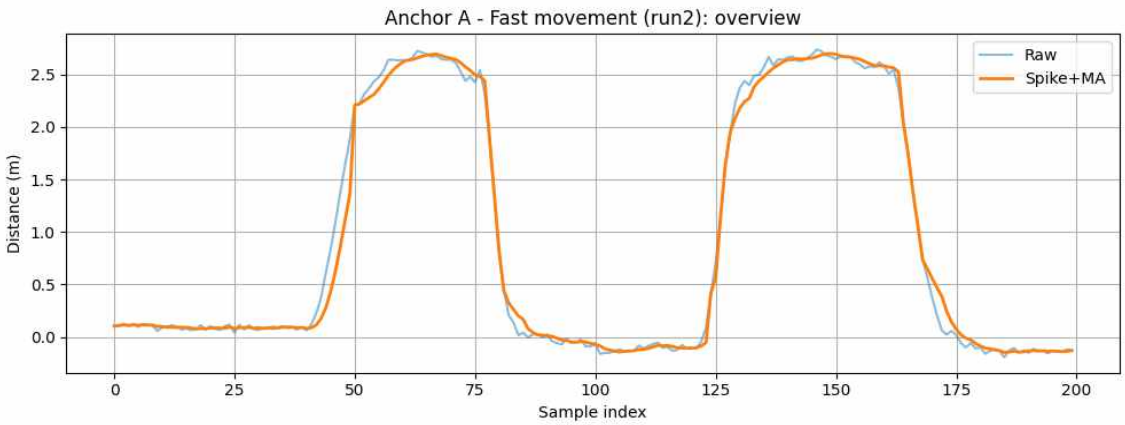
<그림 2.5.3> 앵커 A 확대: 고빈도 잡음 제거 효과 확인



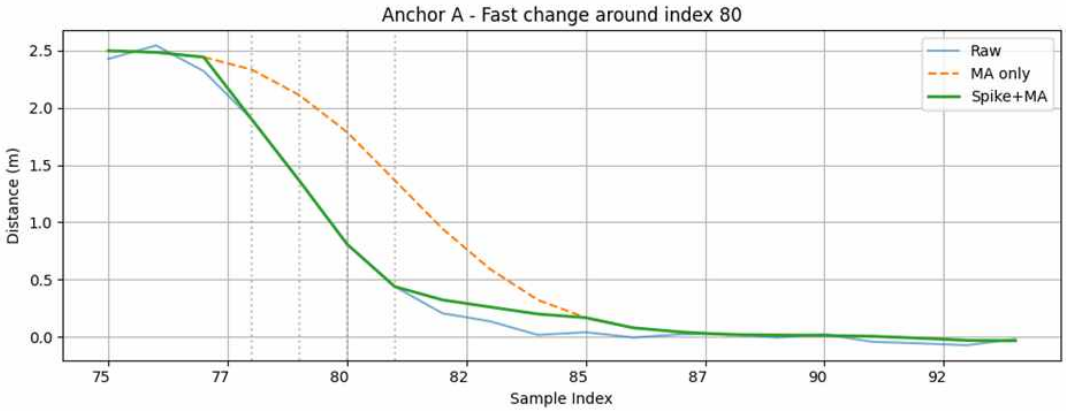
<그림 2.5.4> 앵커 B 확대: 고빈도 잡음 제거 효과 확인

2) 급격한 이동 시 스파이크 필터의 동적 반응성

사용자의 급격한 움직임을 모사하기 위해 태그 위치를 빠르게 변화시키며(Sample Index 40, 80, 120, 160 부근), (1) 원시값 (2) 단순 이동평균 필터 (3) 스파이크 제거 로직이 추가된 제안 필터의 응답 특성을 비교하였다.

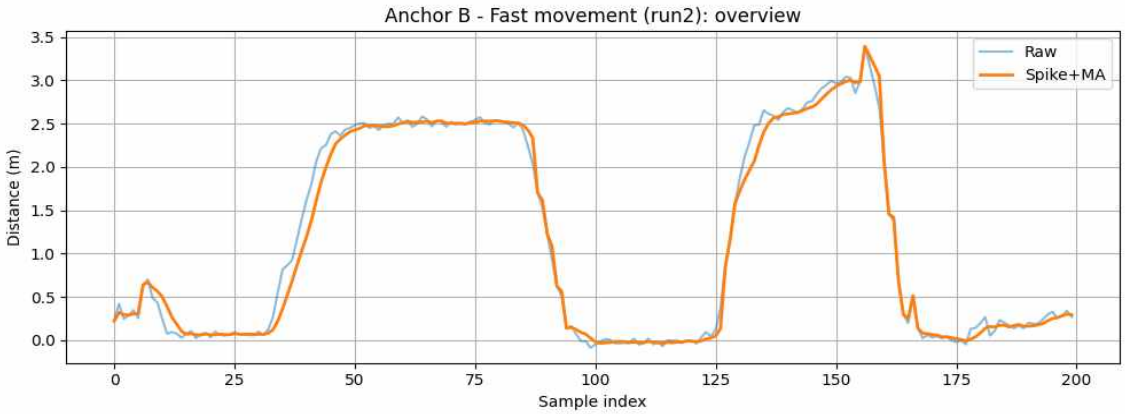


<그림 2.5.5> 앵커 A 동적 실험: 원시값 vs 스파이크 필터 반응성 비교

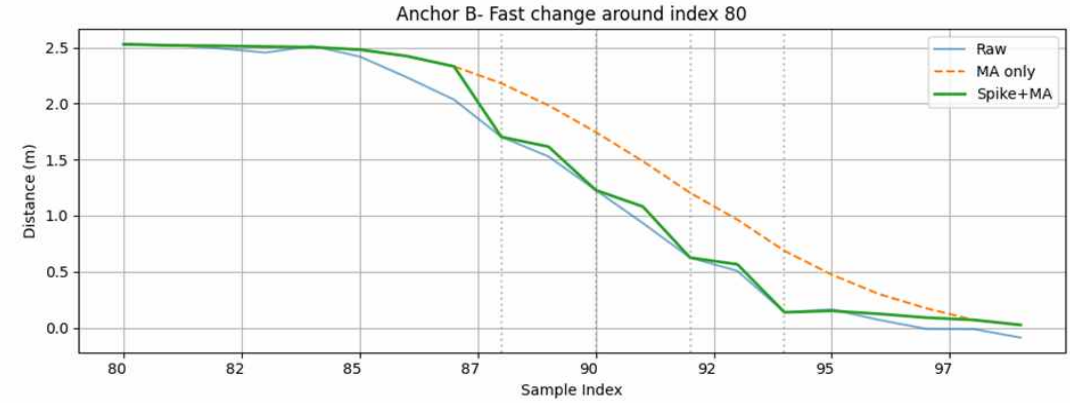


<그림 2.5.6> 앵커 A 동적 실험 변화 구간 확대

<그림 2.5.6> 에서 확인할 수 있듯이, 단순 이동평균 필터는 과거 데이터의 영향으로 약 10~20 샘플 정도의 지연(Delay)이 발생하는 반면, 스파이크 필터는 급격한 변화 감지 시 버퍼를 초기화함으로써 지연 없이 즉각적으로 실제 거리를 추종하는 우수한 동적 반응성을 보였다. 이는 앵커 B(<그림 2.5,7>, <그림 2.5.8> 에서도 동일하게 나타났다.



<그림 2.5.7> 앵커 B 동적 실험: 원시값 vs 스파이크 필터 반응성 비교



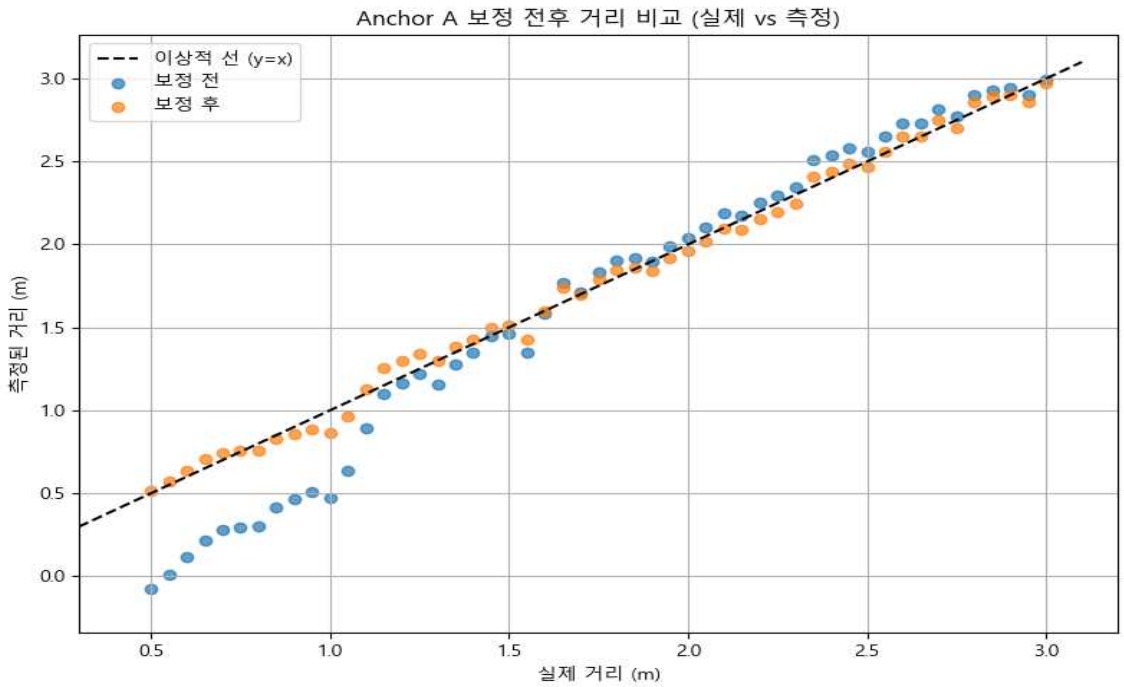
<그림 2.5.8> 앵커 A 동적 실험 변화 구간 확대

(2) 거리 오차 보정 성능

2.4.5절에서 도출한 3차 다항식 보정 모델의 유효성을 검증하기 위해, 실제 측정 거리(Ground Truth)와 보정 전/후 측정값의 오차를 비교 분석하였다.

앵커 A 보정 결과 :

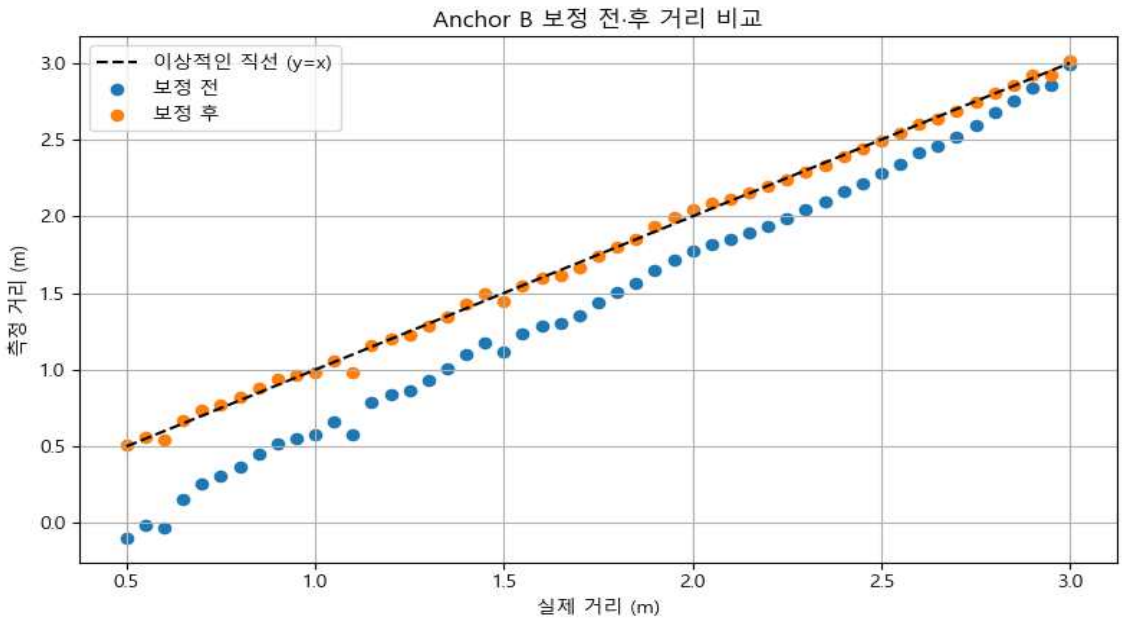
- <그림 2.5.9>는 앵커 A의 보정 전후 거리 비교 그래프이다. 보정 적용 후, 근거리에서의 비선형 오차가 크게 개선되었으며 RMSE(Root Mean Square Error)는 보정 전 0.2428 m에서 0.0750 m로 약 69% 감소하였다.



<그림 2.5.9> 앵커 A 보정 전·후 거리 비교 및 선형성 개선

앵커 B 보정 결과:

- 앵커 B(<그림 2.5.10>)의 경우, 보정 전 0.3325 m였던 RMSE가 0.0288 m로 약 91% 감소하여 매우 높은 보정 효과를 확인하였다.



<그림 2.5.10> 앵커 B 보정 전・후 거리 비교 및 선형성 개선

(3) 삼각측량 기반 위치 추정 정확도

최종적으로 두 앵커의 보정된 거리(d_A, d_B)를 이용하여 삼각측량 알고리즘(2.4.6절)을 수행했을 때, 카트 중심으로부터 태그까지의 거리 추정 정확도를 평가하였다.

정면 거리 측정 정확도 :

- 사용자가 카트 정면에 위치했을 때, 주요 거리 지점(0.6m ~ 3.0m)에서의 추정 오차는 <표 2.5.1>과 같다.
-

<표 2.5.1> 정면 거리 측정 시 실제 거리 대 추정 거리 오차 분석

실제 거리 (m)	추정 거리 (m)	절대 오차	상대 오차
3.00	2.907	0.093 m	3.12 %
2.50	2.441	0.059 m	2.37 %
2.00	1.954	0.046 m	2.29 %
1.50	1.501	0.001 m	0.06 %
1.00	0.890	0.110 m	11.04 %
0.66	0.589	0.071 m	10.80 %

실험 결과, 대부분의 구간에서 5% 이내의 정밀도를 보였으나, 1m 이하 근거리에서는 안테나 간섭 등의 영향으로 약 10% 내외의 오차가 발생하였다. 그러나 이는 0.6m 근접 정지 동작을 수행하는 데에는 충분한 수준으로 판단된다.

각도 편차에 따른 강인성 테스트 :

- 사용자가 정면이 아닌 측면(45도, 80도) 에 위치할 경우의 거리 추정 성능을 <표 2.5.2>에 나타내었다.

<표 2.5.2> 측면 위치(1.2m)에서의 각도별 거리 추정 오차

case (각도)	실제 거리	측정 값	절대 오차	상대 오차
45도	1.2	1.268	0.068 m	5.70 %
80도	1.2	1.122	0.078 m	6.50 %

카트와 태그 사이의 각도가 크게 틀어진 상황에서도 거리 오차는 약 7~8 cm 수준(6% 이내)으로 유지되어, 사용자가 측면으로 이동하더라도 안정적인 거리 추종이 가능함을 검증하였다.

(4) 통합 주행 제어 및 시스템 연동 검증

본 절에서는 앞서 설계한 추종 알고리즘과 장애물 회피 로직이 실제 하드웨어에서 정상적으로 수행되는지, 그리고 해당 상태 정보가 무선 네트워크를 통해 안드로이드 앱에 정확히 반영되는지를 검증하였다.

■ 장애물 회피 및 자율 주행 시퀀스:

구현된 상태 기반 제어(FSM) 로직의 동작을 확인하기 위해, 주행 경로상에 장애물을 배치한 후 카트의 반응을 시계열로 분석하였다. <그림 2.5.11>는 실제 주행 실험 영상을 캡처한 동작 시퀀스이다.



<그림 2.5.11> 장애물 회피 및 추종 주행 동작 시퀀스

- 시퀀스 1 (정상 추종): 사용자를 인식하고 설정된 거리(0.6m) 내에서 안정적으로 추종한다.

- 시퀀스 2 (장애물 탐지 및 정지): 전방 30cm 이내에 장애물이 감지되자, 즉시 모터를 정지하고 회피 모드(Avoid Mode)로 진입한다.

- 시퀀스 3-4 (회피 기동): 후진하여 공간을 확보한 뒤, 초음파 센서 데이터를 기반으로 여유 공간이 있는 방향으로 회전한다.

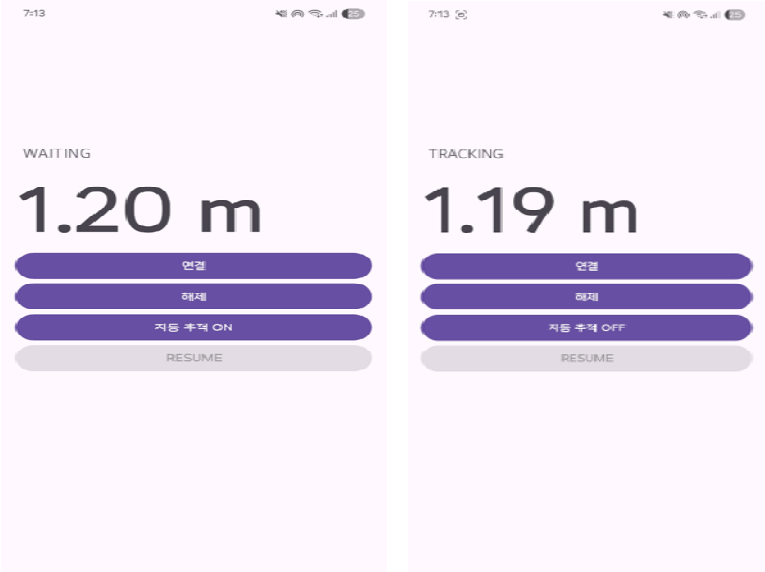
- 시퀀스 5-6 (복귀): 장애물 회피가 완료된 후, 다시 사용자를 향해 방향을 잡고 추종 모드로 자동 복귀하여 주행을 재개한다.

이 실험을 통해 UWB 거리 유지, 초음파 장애물 감지, 그리고 회피 알고리즘이 유기적으로 동작함을 확인하였다.

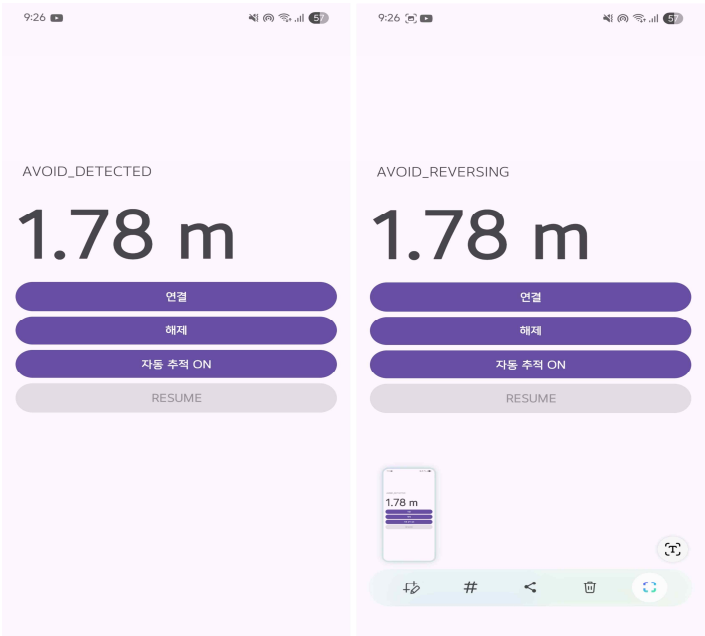
■ 안드로이드 애플리케이션 상태 동기화

다음으로, 스마트 카트의 내부 상태 변화가 TCP/IP 통신을 통해 사용자 인터페이스(App)에 실시간으로 전달되는지를 검증하였다. <그림 2.5.12> 과같이 카트가 대기 중일 때는 WAITING 상태와 현재 거리가 표시되며, 추종 시작 시 TRACKING 모드로 즉시 전환됨을 확인하였다.

또한 <그림 2.5.13> 와 같이 장애물이 감지되는 즉시 앱 화면에 AVOID_DETECTED 상태가 표시되었으며, 카트가 후진하는 동안에는 AVOID_REVERSING으로 상태 메시지가 실시간 갱신되었다. 이를 통해 하드웨어의 FSM 상태 전이가 지연 없이 사용자에게 전달되어, 원격 모니터링 시스템이 신뢰성 있게 동작함을 검증하였다.



<그림 2.5.12> 정상 주행 상태 모니터링 (대기 및 추종)



<그림 2.5.13> 장애물 감지 및 회피 상태 알림

제 3 장. 결론

3.1. 결론

본 연구에서는 UWB 기반 사용자 추종 스마트 카트 시스템을 구현하고, UWB 거리 측정–필터링–보정–삼각측량–추종 제어까지 전 과정을 실제 하드웨어 환경에서 실험적으로 검증하였다. 핵심 실험 결과는 다음과 같다.

첫째, UWB 거리 필터링 성능 실험을 통해 이동평균 필터가 원시 거리값의 고빈도 잡음을 효과적으로 억제함을 확인하였다. 정적 이동 실험에서 원시 데이터 대비 이동평균 필터를 거친 데이터의 변동 폭이 감소함을 확인하였고, 스파이크 감지 필터를 적용하여 급격한 거리 변화 발생 시 이동평균 필터의 지연 문제를 제거하여 즉각적인 거리 갱신을 가능하게 하였다.

둘째, 3차 다항식 거리 보정 실험을 통해 앵커별 비선형 측정 오차가 크게 개선되었다. 실제 거리와의 비교 평가에서 Anchor A는 RMSE 0.2428m → 0.0548m, Anchor B는 RMSE 0.3325m → 0.0288m로 감소하여, 약 70~90% 오차 감소 효과를 확인하였다. 이를 통해 0.5~3.0m 구간에서 cm 단위 정밀도를 확보하였다.

셋째, 삼각측량 기반 거리 측정 실험을 통해 실제 거리 대비 5% 이내의 오차로 사용자와 카트간의 거리 계산이 가능함을 확인하였다. 또한 1.2 m 지점에서 다양한 각도 조건(좌/우 회전) 실험을 수행한 결과 각도 변화에 따른 오차는 6~8 cm 수준으로, 사용자 측면 이동 상황에서도 안정적인 거리 추정이 가능함을 검증하였다.

넷째, 통합 주행 실험을 통해 제안된 상태 기반 제어(FSM) 시스템의 실용성을 검증하였다. 카트는 설정된 목표 거리(0.6m) 이상에서 사용자를 안정적으로 추종하였고, 초음파 센서를 통한 장애물 감지 시 회피 및 재추종 동작이 유기적으로 수행되었다. 또한, 안드로이드 앱과 카트 간의 TCP/IP 통신을 통해 거리 정보와 제어 상태가 지연 없이 실시간으로 동기화됨을 확인하였다.

이와 같은 실험적 검증을 통해 본 연구에서 설계한 스마트 카트 시스템은 정확한 거리 추정, 안정적인 추종 제어, 그리고 실시간 장애물 회피 기능을 갖추어, 실제 환경에서 활용 가능한 자율 추종 플랫폼으로 서의 가능성을 확인하였다.

3.2. 향후 발전

본 연구는 사용자 추종 스마트 카트의 핵심 기능을 구현하고 성능을 검증하였으나, 실제 복잡한 환경에 적용하기 위해 다음과 같은 개선이 필요하다.

■ 하드웨어 및 신호처리 고도화
현재 사용된 DWM1000 모듈은 다중 경로 페이딩에 취약한 한계가 있다. 향후 DWM3000 시리즈 또는 AoA(Angle of Arrival) 기반 위치추정 기술을 도입하여 NLOS(Non-Line-of-Sight) 환경에서의 강인성을 확보할 필요가 있다. 또한 UWB 채널 설정 및 전송 파라미터 최적화를 통해 측정 정밀도를 추가로 향상시킬 수 있다.

■ 주행 제어 알고리즘 개선
현재 카트는 단일 P 제어 기반으로 속도와 조향을 결정하기 때문에, 급격한 방향 전환 시 오버슈트가 발생할 수 있다. 이를 해결하기 위해 PID 제어 또는 모델 예측 제어(MPC)를 도입하면 더욱 부드럽고 정밀한 궤적 추종이 가능하다.

■ 센서 융합 기반의 인지 능력 강화
초음파 센서는 감지 범위가 제한적이며 사각지대가 존재한다. 향후 2D LiDAR 또는 비전 센서(Camera)와의 센서 융합을 적용하면 장애물 인식률을 높일 수 있으며, 단순 회피를 넘어 경로 생성기능까지 확장할 수 있다.

■ 다중 사용자 및 ID 기반 추종 확장
본 시스템은 1인의 사용자를 추종하는 구조이다. 물류, 쇼핑 환경에서 활용도를 높이기 위해서는 UWB 태그 ID 기반 식별 기능을 도입하여 복수의 카트가 혼재한 환경에서도 지정된 사용자만을 정확히 추종하도록 기능을 확장해야 한다.

향후 이러한 센서 융합, 제어 알고리즘 고도화, 하드웨어 개선을 통해 본 연구에서 구현된 스마트 카트 시스템은 실제 쇼핑 및 물류 환경에서도 충분한 안정성과 신뢰성을 갖춘 상용화 가능한 자율 추종 플랫폼으로 발전할 수 있을 것으로 기대된다.

참고문헌

[1] Decawave Ltd., “DWM1000 Datasheet,“ Version 2.10, 2016.

[2] STMicroelectronics, “L298 Dual Full-Bridge Driver,“ Datasheet, Jan. 2000

[3] E. J. Morgan, “HCSR04 Ultrasonic Sensor,“ Datasheet, Nov. 2014.

[4] SparkFun Electronics, “HC-SR04 Ultrasonic Sensor,” Datasheet, Nov. 2014.

[5] G. O. Betancur and F. Treven, “A Framework for a Relative Real-Time Tracking System Based on Ultra-Wideband Technology,“ Master’s thesis, Chalmers Univ. of Technol., Gothenburg, Sweden, 2017.

부록

[1] 소스 코드, 실험 데이터

https://github.com/cej830/TUK_UWBsmartcart